The Critical Path

RL at Scale

A Practical Guide to Reinforcement Learning
for Frontier AI Systems

Amiya Diwan

April 10, 2026

Contents

Introduction

The hardest problems of the next two decades (energy production, semiconductor manufacturing, human healthspan, environmental sustainability) will need better tools and more people working on them. Right now, the on-ramp to contributing at the frontier is unnecessarily steep: years of specialized training, the right credentials, access to the right institutions. This series, The Critical Path, is about lowering that barrier by teaching cross-disciplinary thinking and practical skills that transfer across problem domains.

Why AI Capability Comes First

As of early 2026, frontier AI labs carry significant fixed commitments: multi-year compute rental contracts on the order of $10–13B per year per gigawatt of capacity, against revenue that depends on the variable cost of serving each token. A lab generating $20B in annual revenue with gross margins below 50% has little room between what it earns and what it spends on compute. Growing capacity requires further capital investment that can exceed current revenue. The cost per token is what determines whether the economics work.

AI is a meta-tool: making it smarter accelerates progress on every other problem this series will address. Smarter models come from reinforcement learning. RL from verifiable rewards is how frontier models improve after pretraining, and RL is bottlenecked by rollout generation, which is autoregressive token sampling that consumes 50–70% of training wall-clock time. Rollout generation is an inference workload. More rollouts per dollar produce better gradient estimates, which produce smarter models.

Every technique that reduces inference cost simultaneously improves margins on serving revenue and increases the number of rollouts per dollar of training compute, because rollout generation and production serving are the same workload. That convergence is why inference optimization is the first volume in this series.

We measure every chapter against a single metric:

North Star Metric

Policy improvement per dollar of rollout compute.

Every chapter in this book is evaluated against this metric. A kernel optimization matters if it increases rollout throughput per dollar. A policy gradient variant matters if it extracts more improvement from the same rollouts. Everything else is interesting but off-topic.

Principles

The following four tenets establish how each problem in this book should be approached.

Follow the problem, not the silo. The hardest problems in this field cross the boundary between engineering, ML research, economics, risk and ethics. Readers should care deeply about solving important problems over a specialized title. This is a mindset shift that embraces multi-disciplinary thinking.

Prioritize ruthlessly and test cheaply. Before building infrastructure around an idea, find the riskiest assumption, the one whose failure would make the rest irrelevant, and test it at the

smallest informative scale. A 1M-parameter model on a single GPU can hit the same memory wall as a 70B model on a cluster, provided the experiment isolates that specific effect.

Cultivate taste. You should develop your own neural net alongside using AI. This is hard work and there are no short-cuts, but the reward is that AI becomes your tool, not your crutch. You develop taste through deliberate practice: predict an outcome based on principled reasoning, carefully validate outputs, attempt to explain the results and prioritize the next loop with the highest probability of success. Magnus Carlsen studied AlphaZero's strategies to improve his own chess game, and then tested his learnings against other opponents. The Think First exercises encourage you to develop taste, asking you to commit to an answer before reading ours.

Don't overfit to the stack. Model capabilities are improving fast enough that what counts as a valuable human skill shifts year to year. Two years ago writing Python was scarce; today models write it for you. In RL, a policy trained on a single environment collapses when the dynamics change, and engineering skills work the same way. Master current tools deeply, but extract the transferable reasoning about why systems behave the way they do, so you adapt when the next shift comes. If a specific exercise in this book has been overtaken by a more capable model, you should be able to formulate your own replacement.

Who This Book Is For

This book is written for people who want to work at the intersection of systems engineering and machine learning research, regardless of which side they are coming from.

You might be an ML researcher or research engineer who understands policy gradients but has never profiled a CUDA kernel. You want to know why your training run is slow and whether the answer is algorithmic or architectural.

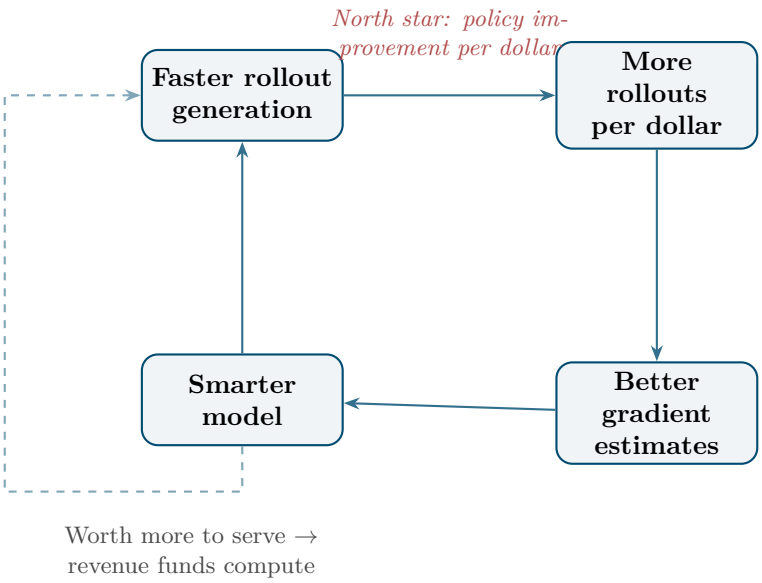
Or you might be a strong software engineer or systems programmer who wants to contribute to frontier AI research. You have the engineering instincts but not the ML background. You do not have a PhD and you do not need one.

Both readers are held back by the same artificial boundary between "systems person" and "research person." This book treats that boundary as a liability, not a specialization.

Prerequisites. We assume you are comfortable with Python and PyTorch, basic linear algebra, and how neural networks are trained. We assume you know what a transformer is, how autoregressive generation works, and what a policy and reward signal are at a conceptual level. We do not assume you know why rollout generation is the bottleneck, what makes a policy gradient estimator high-variance, or how to profile a CUDA kernel.

Format. This is a workbook, not a textbook. Think First exercises ask you to reason about a problem before reading the solution. Exercises are hands-on. Portfolio projects produce artifacts you can show to an employer or collaborator. Passive reading will not get you what this book offers.

The following loop is the system this book teaches you to accelerate. Every chapter targets one edge of this cycle.



The flywheel above is the core feedback loop. Faster rollout generation yields more rollouts per fixed compute budget, which produces lower-variance policy gradient estimates, which produces a smarter model. A smarter model is worth more to serve, and inference revenue funds the next round of compute, which funds further optimization of rollout generation. Every chapter in this book accelerates one edge of this cycle.

Chapter Map

Five content chapters form a linear pipeline. Each builds on the previous, and each teaches a distinct thinking pattern:

Ch	Title	Thinking Pattern
1	The Memory Wall	Reason about hardware
2	Kernel Optimization	Optimize the building blocks
3	Rollout Generation	Optimize the system
4	Policy Gradients	Understand the consumer
5	The RL Pipeline	Build the whole thing

Chapter 1 establishes why inference is memory-bound, not compute-bound, on current hardware. Chapter 2 applies that understanding to individual kernels. Chapter 3 composes kernels into a rollout generation system and optimizes end-to-end throughput. Chapter 4 steps back to understand the consumer of those rollouts: policy gradient algorithms, their variance properties, and what they actually need from the generation system. Chapter 5 wires everything together into a functioning RL training pipeline, closing the loop from hardware to learning signal.

The pipeline is linear, but the understanding is circular. After Chapter 5, you will read Chapter 1 differently.

The Memory Wall & Inference Architectures

Why This Matters at Frontier Scale

RLVR training is an inference workload in disguise. Generating rollouts consumes 50–70% of wall-clock time, because each token requires loading the full weight matrix from DRAM. The bottleneck is not compute. It is memory bandwidth: how fast you can move weights from off-chip memory to the arithmetic units.

Every inference chip designed in the last three years is a different answer to the same question: how do you move data faster, or avoid moving it at all? This chapter gives you the mental model to evaluate any chip announcement, any hardware procurement decision, and any architectural claim against first principles. The metric: policy improvement per dollar of rollout compute.

Move 1: The Memory Wall as First Principles

One fact explains the entire inference hardware landscape.

Gholami et al. (2024) quantified the divergence: compute performance has roughly tripled every two to three years. Off-chip memory bandwidth has improved by a factor of approximately 1.6× over the same intervals. Compute scales exponentially. Memory bandwidth scales linearly. The gap between what the arithmetic units can consume and what the memory system can deliver widens with every generation.

This is the memory wall, a measured trend spanning four decades of semiconductor scaling. Everything that follows in this chapter is a consequence.

Prefill vs. decode: the two regimes. Transformer inference has two distinct phases, and they sit on opposite sides of the roofline.

Prefill processes the entire input prompt in parallel. All tokens are known; the attention computation is a large batched matrix multiply. Arithmetic intensity is high, at $O(N)$ FLOPs per byte, where N is sequence length. Prefill is **compute-bound**. More FLOP/s means faster prefill.

Decode generates one token at a time, sequentially. Each step loads the full weight matrix (billions of parameters × 2 bytes for BF16) from DRAM and performs a single matrix-vector product. Arithmetic intensity is approximately 1–2 FLOP/byte. Decode is **memory-bandwidth-bound**. More TB/s means faster decode.

Every hardware design is a bet on which side of this split to optimize. Training-oriented chips maximize FLOP/s. Inference-oriented chips maximize bandwidth per dollar. The most interesting designs try to collapse the distinction entirely.

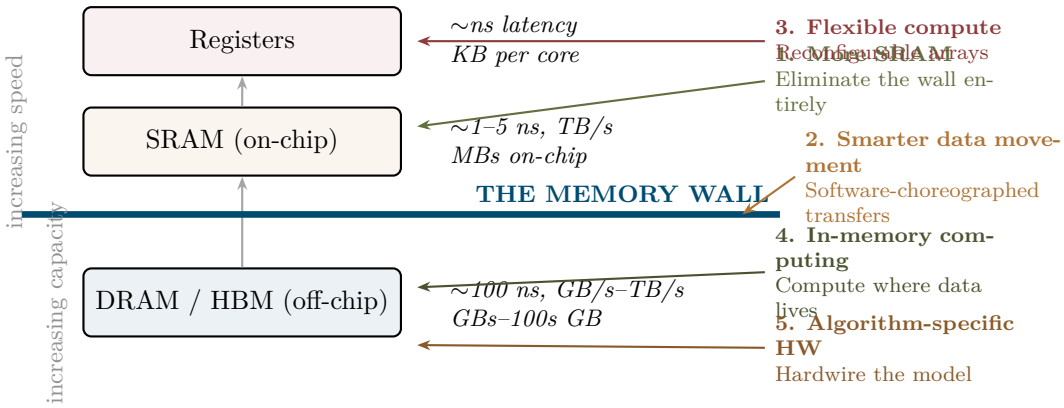
MoE as the extreme illustration. Mixture-of-Experts models amplify the memory wall to its logical extreme. A 400B-parameter MoE with 8 experts and 2 active per token stores 8× the weights but uses only 2× the compute per forward pass. The ratio of parameters loaded from memory to FLOPs performed is 4× worse than a dense model of the same active compute. MoE models are maximum memory pressure per FLOP of useful compute. Any chip that cannot move weights fast enough, or hold enough of them on-chip, pays an outsized penalty on MoE workloads.

Think First (no computer): The Decode Chip

Before reading further, consider: if you were designing a chip specifically for autoregressive decoding, what would you change about the memory system? What tradeoffs would you accept? Think about:

- The arithmetic units are idle > 90% of the time during decode, waiting for data. What if you made them smaller and spent the die area on something else?
- SRAM is ∼5× more expensive per byte than DRAM, but ∼100× faster. At what model size does the cost of putting all weights in SRAM become prohibitive?
- If you could only optimize one of peak FLOP/s, memory bandwidth, or on-chip SRAM capacity, which would reduce decode latency the most for a 70B model?

Write your answer before continuing. The five strategies below are five different answers to this question.



Memory hierarchy with the five architectural strategies mapped to their intervention points. The memory wall (heavy blue line) separates fast on-chip SRAM from slow off-chip DRAM. Each strategy attacks a different part of the problem.

Move 2: Five Architectural Approaches to the Memory Wall

These are not vendor categories. They are *strategies*, distinct engineering bets on how to solve the data movement problem. A single chip may combine elements of multiple strategies. The taxonomy is useful because it tells you what to ask about any new chip: which strategy is it pursuing, and what does it sacrifice?

Strategy	Approach	Tradeoff	Current Example
More SRAM	Eliminate the wall by making on-chip memory enormous	Wafer-scale cost; practical limits for very large models	Cerebras WSE-3 (900K cores, 44 GB SRAM)
Smarter data movement	Software-choreographed access patterns on existing hardware	Depends on software sophistication; limited by SRAM size	Nvidia Blackwell (∼500 MB SRAM)
Flexible compute	Reconfigurable arrays that adapt to prefill vs. decode	Design complexity; utilization depends on workload mix	MatX splittable systolic arrays
In-memory computing	Collapse memory and compute into same substrate	New manufacturing; limited reprogrammability	d-Matrix
Algorithm-specific HW	Strip everything away; hardwire the model architecture	Obsolescence risk: 12–18 mo chip cycle vs. faster algorithm evolution	Etched, CAS wire-encoded weights

Strategy 1: More SRAM. The most direct attack on the memory wall: make the on-chip memory large enough to hold the entire model. If all weights are in SRAM, every decode step reads from fast local memory. No DRAM access, no memory wall.

Cerebras WSE-3 puts 44 GB of SRAM across 900,000 cores on a single wafer-scale die. For models that fit in 44 GB (roughly a 22B-parameter model in BF16, or a 44B model in INT8), decode latency is dominated by compute, not memory bandwidth. The memory wall disappears.

The tradeoff is literal: wafer-scale manufacturing means every defect risks the entire die, yield management is a first-order engineering challenge, and 44 GB is a hard ceiling. A 70B dense model in BF16 does not fit. MoE models with hundreds of billions of total parameters do not fit. The strategy is transformative for models within its capacity and irrelevant for models outside it.

When it wins: Latency-sensitive decode workloads where the model fits in on-chip SRAM. Batch-size-1 inference at minimum latency.

When it loses: Large dense models, MoE models, or workloads that need the SRAM capacity for KV-cache rather than weights.

Strategy 2: Smarter data movement. Accept the memory wall's existence but move data across it as efficiently as possible. This is the GPU approach: large SRAM caches, hardware-managed prefetching, software-level kernel fusion, and tiling strategies that maximize data reuse in fast memory.

Nvidia's Blackwell architecture exemplifies this. The B200 has approximately 500 MB of aggregate on-chip SRAM across all SMs (L1/shared memory + L2 cache). It cannot hold the model, but aggressive kernel design (FlashAttention, fused MLP kernels, continuous batching) minimizes redundant DRAM accesses. The memory wall is not eliminated; it is managed through software sophistication.

The ceiling is SRAM size. No matter how clever the software, if the working set for a single decode step exceeds on-chip capacity, the kernel must go to DRAM. For large batch decode with long KV-caches, the working set grows with sequence length $\times$ batch size. Software optimization has diminishing returns as models and context windows grow.

When it wins: Workloads where software can amortize DRAM accesses across a large enough

batch. High-throughput serving with continuous batching. Hardware ecosystem maturity means battle-tested software stacks.

When it loses: Batch-size-1 latency-sensitive decode, where every step is a cold DRAM fetch regardless of software cleverness.

Strategy 3: Flexible compute allocation. The prefill/decode split is a scheduling problem: prefill wants wide parallel compute, decode wants fast sequential memory access. Most chips are fixed in their compute topology. What if the array could reconfigure itself?

MatX designs splittable systolic arrays that dynamically partition between wide (prefill-optimized) and narrow (decode-optimized) configurations. During prefill, the full array processes the prompt. During decode, the array splits into independent units, each handling a different request with its own memory access pattern.

The tradeoff is design complexity and the assumption that workload heterogeneity is predictable. If your traffic is 90% decode (typical for chat applications), the prefill optimization adds silicon cost without proportional benefit. If your traffic mixes short prompts and long generations unpredictably, dynamic reconfiguration pays off.

When it wins: Mixed workloads with unpredictable prefill/decode ratios. Disaggregated serving architectures that route different phases to different hardware.

When it loses: Homogeneous workloads where you would rather have a simpler, cheaper chip optimized for the dominant phase.

Strategy 4: In-memory computing. The most radical approach: do not move data to compute, but instead compute where the data already lives. In-memory computing embeds arithmetic units directly in or adjacent to the memory arrays, performing multiply-accumulate operations without transferring weights across the memory bus.

d-Matrix builds digital in-memory compute chips where the weight matrix is stored in SRAM bitcells that double as multiply-accumulate units. A matrix-vector product is computed by activating a row of the memory array, with the computation happening as a side effect of the read operation.

The tradeoff is manufacturing maturity and reprogrammability. Analog in-memory computing approaches (using memristors or ReRAM) promise even higher density but suffer from noise and limited precision. Digital approaches avoid the noise problem but sacrifice some of the density advantage. Both approaches make it difficult to update weights dynamically, since fine-tuning or LoRA adapters require rewriting the memory array, which is orders of magnitude slower than DRAM writes.

When it wins: Inference-only deployments with fixed model weights, high volume, and extreme cost-per-token pressure.

When it loses: Any workload that requires frequent weight updates, including RL training loops, where the model is updated every few hundred to few thousand steps.

Strategy 5: Algorithm-specific hardware. If you know exactly what computation the chip will perform, you can strip away all general-purpose overhead. No instruction decode, no branch prediction, no cache hierarchy, just hardwired dataflow for one specific algorithm.

Etched's Sohu chip hardwires transformer inference: the attention pattern, the MLP structure, the layer norm, the residual connections are all fixed in silicon. CAS goes further, encoding model weights directly in the chip's wire routing at fabrication time.

The performance ceiling is extraordinary: no overhead means every transistor contributes to useful

compute. The obsolescence risk is equally extraordinary. The transformer architecture has been stable since 2017, but the specifics of attention (multi-head, grouped-query, multi-latent, linear) change every 6–12 months. A chip that hardwires multi-head attention cannot run grouped-query attention without wasting silicon. The 12–18 month chip design cycle must bet on architectural stability that the research community has not guaranteed.

When it wins: High-volume inference of a specific, stable architecture at extreme cost efficiency.

When it loses: Algorithmic shifts that obsolete the hardwired assumptions. Research workloads that require architectural flexibility.

Research Taste Check

When you read about a new chip announcement, your first question should be: which of these five strategies is it pursuing? Your second: what does it sacrifice?
The press release will emphasize the strategy's strengths. Your job is to identify the tradeoff. A chip that "eliminates the memory wall" with large SRAM: at what model size does it stop fitting? A chip with "10× better cost-per-token" through algorithm-specific hardware: what happens when the architecture changes? A chip with "breakthrough in-memory computing": can you fine-tune on it?
The strategies are not ranked. Each wins under different conditions. The skill is matching the strategy to the workload.

Move 3: Decision-Grade Benchmarking on Unfamiliar Hardware

Reading spec sheets tells you what a chip *could* do. Benchmarking tells you what it *does* do on your workload. The gap between these two numbers is where engineering effort lives.

This section provides a hardware-agnostic benchmarking template. The template is the repeatable skill; the two exercise instances that follow are applications of it.

The template: six steps from spec sheet to procurement decision.

1. **Identify peak specs.** Find the hardware's theoretical peak compute (FLOP/s at your target precision) and peak memory bandwidth (TB/s). These are the roofline ceilings. If the vendor does not publish both numbers, that tells you something: they are hiding the weaker metric.
2. **Calculate the roofline ridge point.** The ridge point is peak compute divided by peak bandwidth:
$$I^* = \frac{\text{Peak FLOP/s}}{\text{Peak Bandwidth (B/s)}}$$
Operations with arithmetic intensity below I^* are memory-bandwidth-bound. Operations above I^* are compute-bound. The ridge point is the single most informative number about a chip's design philosophy.
3. **Predict where your workload falls.** For RL rollout generation (autoregressive decode, batch sizes 1–64):
   - Each decode step loads the weight matrix ($O(D^2)$ bytes) and performs a matrix-vector product ($O(D^2)$ FLOPs per token in the batch).
   - Arithmetic intensity $\approx B/2$ FLOP/byte for batch size B in BF16 (2 bytes per weight, 2 FLOPs per multiply-accumulate, B tokens amortizing the weight load).
   - At batch size 1: ∼1 FLOP/byte. At batch size 64: ∼32 FLOP/byte. Almost always below the ridge point on modern hardware. Decode is memory-bandwidth-bound.
4. **Measure actual throughput.** Run the workload. Measure tokens/second at target batch

sizes. Compute achieved bandwidth: bytes loaded per second = (model size in bytes) × (tokens/second / batch size). Compare achieved bandwidth to peak bandwidth. The ratio is your bandwidth utilization.

5. **Compute cost-per-token.** Divide hourly hardware cost by measured tokens per second. This is the number that determines procurement decisions, not FLOP/s, not bandwidth, not peak anything, but cost per useful output token.
6. **Produce a one-page comparison.** Against a reference platform (typically H100 SXM), show: peak specs, ridge point, predicted regime, measured throughput, utilization, and cost-per-token. One table, one roofline plot, one recommendation sentence.

The template is designed to take 4–8 hours per platform for a practitioner with SSH access. Most of that time is environment setup and driver installation. The measurements themselves take minutes.

Exercise: AMD MI300X Benchmarking

Hypothesis: MI300X has 5.3 TB/s HBM3 bandwidth vs. H100's 3.35 TB/s, a 1.58× advantage. For memory-bandwidth-bound decode, this should translate to approximately 1.58× generation throughput.

Apply the template:

Step 1 — Peak specs.

- MI300X: 383 TFLOP/s BF16, 5.3 TB/s HBM3 bandwidth, 192 GB capacity.
- H100 SXM: 989 TFLOP/s BF16 (with sparsity; ∼495 dense), 3.35 TB/s HBM3, 80 GB.

Step 2 — Ridge point.

$$I^*_{\text{MI300X}} = \frac{383 \times 10^{12}}{5.3 \times 10^{12}} \approx 72 \text{ FLOP/byte}$$

$$I^*_{\text{H100}} = \frac{495 \times 10^{12}}{3.35 \times 10^{12}} \approx 148 \text{ FLOP/byte (dense)}$$

MI300X becomes compute-bound at lower arithmetic intensity. For intermediate intensities (72–148 FLOP/byte), MI300X may deliver higher absolute throughput despite lower peak FLOP/s.

Step 3 — Predict. Autoregressive decode at batch size 1–64: arithmetic intensity ≈ 0.5–32 FLOP/byte. Well below both ridge points. Both chips are memory-bandwidth-bound. Predicted speedup: 5.3/3.35 = 1.58×.

Steps 4–6 — Measure. Rent MI300X on Crusoe or TensorWave (∼$1.70/hr). Load a 7B model in BF16. Generate 512 tokens at batch sizes $\{1, 4, 8, 16, 32, 64\}$. For each batch size, record:

- Tokens/second (end-to-end, including KV-cache updates).
- Achieved HBM bandwidth: (model bytes × tokens/sec / batch size) / 5.3×10^{12}.
- Cost per 1M tokens at measured throughput.
- Ratio vs. H100 baseline (use published vLLM benchmarks or your own measurements).

Diagnostic: If the measured speedup is $< 1.3\times$, the gap is not in HBM bandwidth but in software. Profile with `omniperf` to identify whether the bottleneck is attention kernel efficiency, RCCL initialization overhead, or KV-cache management. The MI300X has 8 XCDs connected via Infinity Fabric; inter-XCD communication for KV-cache sharding can erase the bandwidth advantage if the attention kernel does not account for the topology.

Record: Roofline plot with both chips, measured throughput table, utilization percentages, cost-per-token comparison, one-paragraph recommendation.

Exercise: Cerebras Cloud Benchmarking

Hypothesis: Models fitting in Cerebras WSE-3's 44 GB SRAM should show dramatically lower decode latency than GPU-based systems, because decode becomes compute-bound rather than memory-bandwidth-bound when weights are in SRAM. There exists a crossover model size beyond which Cerebras loses its advantage.

Apply the template:

Step 1 — Peak specs. Cerebras WSE-3: 900,000 cores, 44 GB on-chip SRAM, internal SRAM bandwidth in the hundreds of PB/s range (Cerebras claims 21 PB/s on-chip). Peak compute approximately 125 PFLOP/s (INT8). The memory wall effectively does not exist for models that fit on-chip.

Step 2 — Ridge point. The roofline model does not apply in the conventional sense. When weights are in SRAM, the bandwidth ceiling is so high that all operations are compute-bound. The relevant question is not "what is the ridge point" but "does the model fit in 44 GB?"

Step 3 — Predict.

- A 22B-parameter model in BF16: $22 \times 10^9 \times 2 = 44$ GB. Just fits. Decode should be dramatically faster than any GPU.
- A 7B-parameter model in BF16: 14 GB. Fits easily. All 30 GB of remaining SRAM is available for KV-cache (∼60K tokens for a 32-layer model at 512 KB/token).
- A 70B-parameter model in INT8: 70 GB. Does not fit. Must use external DRAM, which reintroduces the memory wall.

Steps 4–6 — Measure. Use Cerebras Cloud API. Benchmark three model sizes that bracket the 44 GB boundary: 7B (fits easily), 22B (fits tightly), and 70B (does not fit). For each:

- Time-to-first-token (TTFT) at batch size 1.
- Tokens/second during decode.
- Cost per 1M tokens at Cerebras Cloud pricing.
- Compare each metric to vLLM on H100.

Key question: At what model size does the Cerebras advantage disappear? Plot tokens/second as a function of model size for both Cerebras and H100. The crossover point is the decision boundary: below it, Cerebras wins on latency; above it, GPUs win on cost-efficiency.

Record: Throughput vs. model size plot, latency comparison table, cost-per-token at each model size, identification of the crossover point.

Portfolio Project: Decision-Grade Hardware Benchmarking Report

Deliverable: A publishable benchmarking report that applies the six-step template to at least two platforms.

Structure: The two exercises above produce the raw material. The portfolio artifact is the synthesis: a single document that a technical decision-maker could use to justify a hardware procurement choice. The report should include:

1. Roofline analysis showing where RL rollout generation falls on each platform.
2. Measured throughput at production-relevant batch sizes.
3. Cost-per-token comparison at measured (not theoretical) throughput.
4. A clear recommendation with stated assumptions and conditions under which the recommendation reverses.

Why this is portfolio-quality: Hardware benchmarking reports that combine roofline predictions with measured results, cost analysis, and conditional recommendations are rare. Most

published benchmarks report peak throughput without cost normalization or workload-specific analysis. A report that answers "which chip should I rent for RL rollout generation at batch size 32, and under what conditions does the answer change?" is immediately useful to any team making procurement decisions.
Verification: Show the report to someone who has made a hardware procurement decision. Ask: "Is there enough information here to act on?" If the answer is no, identify what is missing and add it.

Kernel Optimization for Inference

Why This Matters at Frontier Scale

Chapter 1 established the memory wall as the governing constraint on inference hardware. This chapter makes it operational. The roofline model becomes a diagnostic tool applied to specific kernels (attention, quantized matmuls, MoE routing) with concrete numbers on concrete hardware. The question shifts from "why is decode slow?" to "which kernel is the bottleneck, what regime is it in, and what optimization moves it to a faster regime?"

Three to five kernels dominate RLVR wall time. Each sits at a specific point on the roofline. Optimizing the wrong kernel, or the right kernel for the wrong bottleneck, wastes engineering weeks. The skill this chapter builds: diagnose first, optimize second, verify third. The tool: the roofline model, applied with numbers.

Move 1: The Roofline Model as Diagnostic Tool

Chapter 1 introduced the roofline conceptually: operations are either compute-bound or memory-bound, and the ridge point separates the two regimes. Here we make it precise enough to use.

The model. Every kernel has an arithmetic intensity:

$$I = \frac{\text{FLOPs performed}}{\text{bytes moved between DRAM and on-chip memory}}$$

Every chip has a ridge point:

$$I^* = \frac{\text{peak compute (FLOP/s)}}{\text{peak bandwidth (B/s)}}$$

If $I < I^*$, the kernel is **memory-bound**: performance scales with bandwidth. If $I > I^*$, the kernel is **compute-bound**: performance scales with FLOP/s. The attainable throughput is:

$$\text{Throughput} = \min\left(\text{peak FLOP/s},\ \ I \times \text{peak bandwidth}\right)$$

This is a ceiling, not a guarantee. Real kernels hit 50–80% of the roofline ceiling. The gap comes from imperfect tiling, bank conflicts, uncoalesced memory accesses, and pipeline bubbles. But the roofline tells you which ceiling you are hitting, and therefore which class of optimization can help.

Key insight: the same kernel, different regimes. A batched matrix multiply at batch size 1 has arithmetic intensity $\approx$ 1 FLOP/byte (BF16). At batch size 64, intensity rises to $\approx$ 32 FLOP/byte. On an H100 with ridge point $\approx$ 148 FLOP/byte, both are memory-bound. On a chip with ridge point $\approx$ 30 FLOP/byte, batch-64 is compute-bound while batch-1 is memory-bound. The optimization strategy differs completely, and you cannot know which applies without computing the numbers for your specific hardware.

Reference hardware. We use two chips throughout this chapter for concrete examples:

Chip	BF16 Peak (TFLOP/s)	BW (TB/s)	Ridge (FLOP/byte)
H100 SXM (dense)	495	3.35	148
B200	4500	8.0	562

Think First (no computer): Autoregressive Decode Arithmetic Intensity

For autoregressive decode with batch size 1, estimate the arithmetic intensity of a single transformer layer's attention computation. Assume head dimension $d = 128$, 32 heads, and context length $S = 2048$.

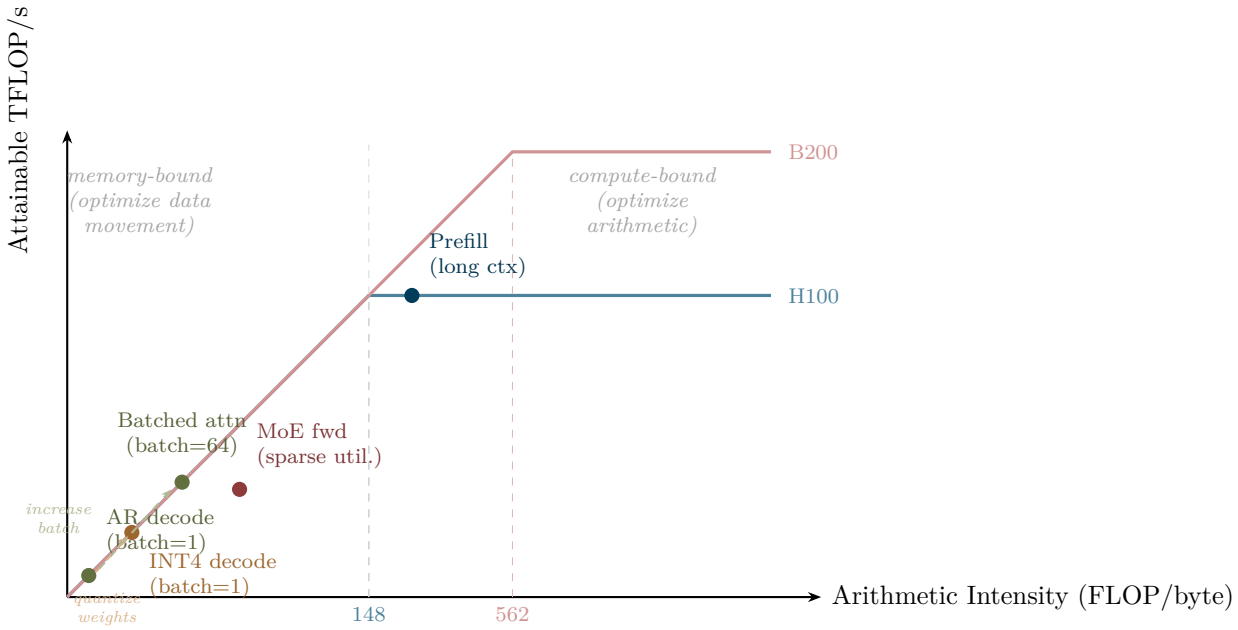
At each decode step, the query is a single vector of dimension d per head. The KV-cache holds S key vectors and S value vectors per head, each of dimension d, in BF16. For one head:

- $QK^\top$: multiply $[1 \times d]$ by $[d \times S] = 2dS$ FLOPs. Read Q ($2d$ bytes) and K cache ($2dS$ bytes).
- Softmax: $O(S)$ FLOPs, negligible relative to matmul.
- scores $\times V$: multiply $[1 \times S]$ by $[S \times d] = 2dS$ FLOPs. Read V cache ($2dS$ bytes).
- Total per head: $4dS$ FLOPs, $\approx 4dS$ bytes read (dominated by KV-cache loads).

Arithmetic intensity per head $\approx 4dS/4dS = 1$ FLOP/byte.

On a chip with 1000 TFLOP/s peak compute and 3 TB/s bandwidth, the ridge point is ≈ 333 FLOP/byte. At intensity 1, this is deep in the memory-bound regime, where throughput is $1 \times 3 \times 10^{12} = 3$ TFLOP/s, using 0.3% of peak compute.

This is why autoregressive decode is the bottleneck: the arithmetic units are idle $> 99\%$ of the time, waiting for data. No amount of faster compute fixes this. Only bandwidth or data movement reduction helps.



Roofline diagram with inference kernels. Autoregressive decode sits deep in the memory-bound regime. Quantization and batching shift operations rightward along the x-axis. Prefill attention reaches the compute-bound regime on H100, but not on B200 (higher ridge point). MoE forward passes underperform their theoretical intensity due to sparse expert utilization and dispatch overhead.

Reading the diagram. Two operations produce the same measured TFLOP/s but for different reasons: AR decode at batch=1 hits the bandwidth ceiling at low intensity; prefill attention hits the compute ceiling at high intensity. The optimization for each is completely different. For decode, reduce bytes moved (quantization, KV-cache compression). For prefill, increase compute

efficiency (larger tiles, better MXU/Tensor Core utilization). Applying the wrong optimization wastes engineering time.

Quantization shifts the x-axis. Moving weights from FP16 (2 bytes) to INT4 (0.5 bytes) reduces bytes loaded by 4× without changing FLOPs. Arithmetic intensity quadruples. This is the primary mechanism by which quantization accelerates memory-bound kernels because it is a bandwidth optimization, not a compute optimization. The FP4 compute throughput increase on Blackwell is a secondary benefit.

Batching shifts the x-axis. At batch size B, the weight matrix is loaded once and used for B matrix-vector products. Arithmetic intensity scales linearly with B. This is why continuous batching (Chapter 3) matters: higher effective batch size moves decode closer to the ridge point.

Move 2: The Critical Kernels, Organized by Bottleneck

The kernels that dominate RLVR inference wall time fall into three categories: attention kernels (the KV-cache problem), quantized matmuls (the weight loading problem), and MoE routing (the sparse dispatch problem). Each has a characteristic roofline position and a characteristic optimization strategy.

2a: Attention Kernels

Why naive attention is unacceptable. Standard attention computes $O = \text{softmax}(QK^\top/\sqrt{d})\,V$. For sequence length N and head dimension d, the intermediate score matrix $QK^\top$ is $N \times N$. Materializing it in DRAM costs $O(N^2)$ bytes. At N=8192 in FP16: 128 MB per head, per layer. A 32-head, 32-layer model needs ∼130 GB of intermediate storage, exceeding a single H100's HBM.

The problem is not the FLOPs (attention is $O(N^2 d)$). The problem is materializing the N^2 matrix in DRAM, reading it back for softmax, then reading it again for the value projection. Three passes over $O(N^2)$ data when DRAM bandwidth is the bottleneck.

FlashAttention: fuse everything, materialize nothing. FlashAttention processes Q, K, V in tiles that fit in SRAM. For each tile of queries, it streams through all key-value tiles, computing attention scores and accumulating the output without ever writing the full $N \times N$ matrix to DRAM.

The key algorithmic insight is the *online softmax*: standard softmax requires a full pass to find the maximum (for numerical stability), then a second pass to exponentiate and normalize. The online algorithm maintains running statistics (m, ℓ), the current maximum and current sum, updating both as new tiles arrive. This enables single-pass softmax over blocks that never coexist in SRAM simultaneously.

HBM traffic drops from $\Theta(N^2 + Nd)$ (standard) to $\Theta(Nd)$ (FlashAttention), linear in sequence length. At $N = 8192$ and $d = 128$, this is a 64× reduction in memory traffic. The compute is identical; only the data movement changes. This is a pure memory-hierarchy optimization.

Decode vs. prefill: two regimes, one kernel. The same attention operation sits in different roofline regimes depending on context:

Prefill processes the full input sequence in parallel. Q, K, V are all $[N \times d]$ matrices. The attention computation is a batched matmul with arithmetic intensity $O(N/d)$. At $N = 4096$ and $d = 128$, intensity ≈ 32 FLOP/byte, approaching compute-bound on some hardware.

Decode generates one token at a time. Q is $[1 \times d]$; the kernel reads the full KV-cache ($[S \times d]$ for each of K and V, where S is the current sequence length). This is a matrix-vector product: arithmetic intensity ≈ 1 FLOP/byte, deep in the memory-bound regime regardless of hardware.

Decode attention is the single most bandwidth-hungry operation in autoregressive generation.

The engineering consequence: decode attention benefits from bandwidth optimizations (KV-cache quantization, larger batch sizes). Prefill attention benefits from compute optimizations (better tiling, higher Tensor Core utilization). A single FlashAttention implementation must handle both regimes efficiently.

Paged attention for KV-cache. Naive KV-cache allocation reserves a contiguous memory block per sequence at the maximum context length. For RLVR rollouts with high length variance (100–4000 tokens), this wastes memory proportional to the gap between actual and maximum length.

Paged attention (vLLM) manages KV-cache as fixed-size blocks allocated on demand and mapped to non-contiguous physical memory, analogous to OS virtual memory pages. Fragmentation drops from $O(S_{\max})$ to $O(\text{block size})$ per sequence. This directly increases the number of concurrent rollouts that fit in HBM, which increases effective batch size, which moves decode attention rightward on the roofline.

KV-cache quantization. FP8 or INT8 quantization of the KV-cache reduces memory by 2× (from FP16) and bytes loaded per decode step by the same factor. The arithmetic intensity of decode attention doubles. For RLVR, this requires caution: KV-cache quantization introduces noise in attention scores, which propagates to generation quality, which directly affects the reward signal and gradient estimates. Unlike supervised training where a small quality degradation is tolerable, RL amplifies distributional shifts, so a 2% quality drop may produce a disproportionate change in the reward distribution, leading to noisier advantage estimates and slower policy improvement. Measure reward variance, not just perplexity, before deploying KV-cache quantization in an RL pipeline.

Exercise: FlashAttention Roofline Profiling

Profile FlashAttention decode latency across batch sizes 1, 8, 32, 64. For each batch size, compute:

- Achieved HBM bandwidth (bytes loaded / latency).
- Arithmetic intensity (FLOPs / bytes loaded).
- Attainable TFLOP/s from the roofline model.

Plot all four points on the roofline diagram for your hardware. Identify the crossover batch size where decode transitions from memory-bound to compute-bound. On H100 (ridge ≈ 148 FLOP/byte), expect this crossover to be unreachable at practical batch sizes. Verify this.
Use a 7B model, sequence length 2048, FP16. Warm up 10 iterations, measure 100, report median. If achieved bandwidth is below 60% of peak at batch size 1, profile memory access patterns; non-coalesced KV-cache reads from paged attention can cause this.

2b: Quantized Matmuls

During autoregressive decode, the weight matmul loads the full parameter matrix from DRAM at every step: for a 7B model in BF16, that is ∼14 GB per step. At H100's 3.35 TB/s, the minimum time per step is 14/3350 ≈ 4.2 ms, just from weight loading. This sets a hard floor on decode latency that no compute optimization can break.

Weight-only quantization. Reducing weight precision from FP16 to INT8 halves the bytes loaded: 7 GB per step, minimum latency ≈ 2.1 ms. INT4 quarters it: 3.5 GB, minimum ≈ 1.0 ms. The FLOPs are unchanged (dequantization to FP16/FP32 for the actual multiply-accumulate is negligible), but the bandwidth requirement halves or quarters. This is the mechanism: quantization

is a bandwidth optimization for memory-bound kernels.

Arithmetic intensity by precision. For an $N \times N$ GEMM:

Precision	Bytes/element	Data loaded	Intensity ($N/\ldots$)	H100 bound ($N = 4096$)
FP16	2	$6N^2$ bytes	$N/3$	compute ($I = 1365$)
FP8	1	$3N^2$ bytes	$2N/3$	compute ($I = 2731$)
FP4	0.5	$1.5N^2$ bytes	$4N/3$	compute ($I = 5461$)

For large square GEMMs (training), all precisions are compute-bound on H100. The benefit of lower precision is more FLOP/s from the Tensor Cores, not less bandwidth.

For decode (effective $N \approx$ batch size $\times\, d$, typically $\ll 4096$): all precisions are memory-bound. The benefit of lower precision is fewer bytes loaded, which is a bandwidth optimization.

FP4 on Blackwell. Blackwell's B200 offers ∼18,000 TFLOP/s at FP4 with 8 TB/s bandwidth. Ridge point: $18000/8 = 2250$ FLOP/byte. For a decode-step weight matmul at batch size 1 with FP4 weights: intensity ≈ 2 FLOP/byte. Still deep in the memory-bound regime. Even Blackwell's enormous FP4 compute advantage does not change this: at batch size 1, the weight load dominates regardless of precision. The 8 TB/s bandwidth is what matters, not the 18,000 TFLOP/s.

The RL tension. Quantization makes rollouts cheaper but noisier. The critical question for RLVR is not "does quantization degrade perplexity?" but "does quantization degrade the reward signal enough to slow policy learning?" These are different measurements. RL amplifies small distributional shifts: a quantized model that generates slightly different token distributions produces different reward statistics, which produce different advantage estimates, which produce different gradients. A 2% perplexity degradation might be invisible in supervised evaluation but produce measurably higher gradient variance in RLVR.

The practical protocol: run a short RL training ablation (100–500 gradient steps) comparing FP16 and quantized rollout generation. Compare reward curves, gradient norm variance, and KL divergence from the reference policy. If gradient variance increases by $> 20\%$ under quantization, the cheaper rollouts are costing you convergence speed.

Think First (no computer): Quantization Acceptability for RL

If you quantize model weights from FP16 to INT4 and rollout quality degrades by 2% on an eval benchmark, is this acceptable for RL training? What would you need to measure to answer this question?

The eval benchmark measures generation quality on a fixed distribution. RL training cares about the reward signal's informativeness for policy improvement. Consider:

- Does the 2% quality drop change which rollouts receive high vs. low rewards? If the reward model assigns similar scores to FP16 and INT4 outputs, the advantage estimates are similar and quantization is fine.
- Does the distributional shift from quantization interact with the reward function? For binary rewards (correct/incorrect on math problems), a small quality drop may flip some borderline problems from correct to incorrect, changing the reward signal discontinuously.
- Does the gradient variance increase? Even if mean reward is similar, higher variance in the advantage estimates slows convergence.
- Does the KL divergence between the quantized policy and the reference policy differ from the FP16 case? GRPO penalizes KL divergence; quantization-induced distributional shift

adds an uncontrolled source of KL that may interact with the KL penalty.

The answer is: you cannot determine acceptability from an eval benchmark alone. You need to measure reward variance, gradient norm statistics, and convergence speed directly under RL training conditions.

2c: MoE Routing and Expert Kernels

Mixture-of-Experts models replace the dense FFN with E parallel expert FFNs, routing each token to the top-k experts via a learned gating function. This introduces kernel-level challenges that do not exist in dense models.

The forward pass. At each MoE layer, the router network (a small linear layer) computes gating scores for all E experts, selects the top-k per token, dispatches tokens to the selected experts, computes the expert FFN outputs, and scatters results back. For a batch of B tokens:

1. **Route:** compute $[B \times E]$ gating scores, select top-k per row. Compute-cheap, memory-cheap.
2. **Dispatch:** gather tokens into per-expert batches. Each expert receives a variable number of tokens. This is an irregular scatter operation.
3. **Compute:** run k expert FFNs. Each expert's batch size varies: some experts may receive many tokens, others few or none.
4. **Combine:** scatter expert outputs back to the original token positions and combine (weighted sum by gating scores).

Load balancing. If the router consistently sends tokens to the same subset of experts, unused experts waste SRAM and compute. A model with 8 experts and 2 active per token but 80% of tokens routed to experts 0 and 1 is effectively a dense model with 6 dead expert FFNs occupying HBM. The auxiliary load-balancing loss encourages uniform routing, but under RLVR training, the hidden state distribution shifts with each policy update, creating transient load imbalances that the loss has not yet corrected.

Expert parallelism. When experts are distributed across GPUs (each GPU holds $E/\text{num\_gpus}$ experts), every MoE layer requires all-to-all communication: tokens must be sent to the GPU holding their selected expert, processed, and sent back. For a model with $L/2$ MoE layers (alternating dense and MoE), this is L all-to-all operations per autoregressive step. At large batch sizes, this communication can dominate step latency.

Sparse dispatch kernels. The gather-compute-scatter pattern is challenging to implement efficiently. Naive implementations use `torch.index_select` for gathering and `scatter_add` for combining, each a separate kernel launch with its own DRAM round-trip. Expert batch sizes are irregular: the GEMM for an expert receiving 3 tokens is a very different shape from one receiving 300 tokens. Padding to a uniform expert batch size wastes compute; dynamic-shape GEMMs underutilize Tensor Cores at small sizes.

Roofline position. MoE forward passes have lower *effective* arithmetic intensity than their theoretical intensity suggests. The theoretical intensity accounts for k expert FFNs, each performing standard GEMMs. The effective intensity is reduced by: dispatch overhead (irregular memory accesses for token gathering), load imbalance (idle experts waste allocated compute), and communication overhead (all-to-all for expert parallelism). On the roofline diagram, MoE operations plot below and to the left of where a dense model with the same active parameter count would sit.

Move 3: Cross-Platform Pattern Matching

The optimization principles from Move 2 apply to every accelerator. The specific APIs and hardware abstractions differ, but the diagnostic thinking is identical: find the bottleneck on the roofline, optimize data movement, overlap compute and memory access. This section maps the concepts between GPU (CUDA) and TPU (Pallas/XLA) to make the universality concrete.

GPU: threads and memory access patterns. On a GPU, thousands of lightweight threads execute the same instruction on different data (SIMT). The programmer manages *shared memory*, fast on-chip SRAM visible to all threads in a thread block. The optimization loop:

1. Tile the computation so each thread block's working set fits in shared memory.
2. Load tiles from HBM to shared memory with coalesced accesses (consecutive threads access consecutive addresses).
3. Compute on the tile, reusing data from shared memory.
4. Use warp-level primitives (`mma.sync`, shuffle instructions) to coordinate within a 32-thread warp without shared memory round-trips.

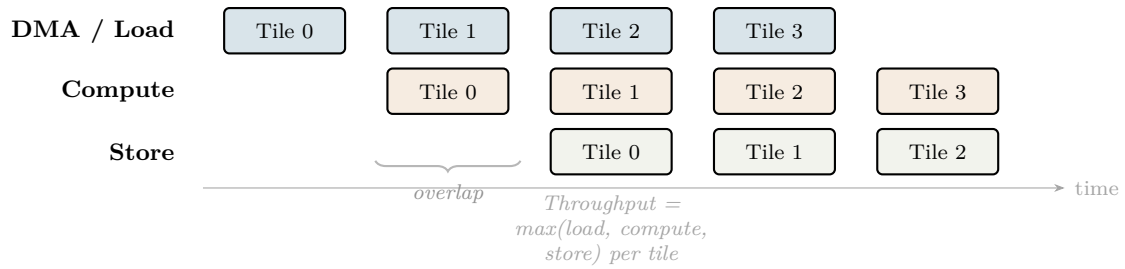
TPU: pipeline efficiency and keeping the systolic array fed. On a TPU, a 128×128 systolic array (MXU) flows data through in a deterministic pattern. There is no warp scheduling randomness. The programmer manages *VMEM* (on-chip SRAM, ∼30 MB on v5e) and *DMA transfers* between HBM and VMEM. The optimization loop:

1. Tile the computation so each tile's inputs fit in VMEM.
2. Prefetch the next tile via DMA while the current tile is being computed on the MXU.
3. Match tile dimensions to the MXU size (128×128). Undersized tiles underutilize the systolic array.
4. Use double-buffering (`Buffered(buffer_count=2)` in Pallas) to overlap DMA and compute.

The mapping. The concepts are the same; only the vocabulary changes:

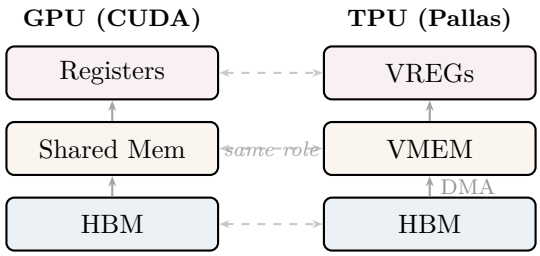
GPU (CUDA)	TPU (Pallas/XLA)
Shared memory tiling	VMEM prefetching
Warp-level primitives (`mma.sync`)	Sublane operations on MXU
Thread blocks	Tiles (grid cells)
Double-buffering in shared memory	`Buffered(buffer_count=2)` DMA
Bank conflict avoidance	VMEM access pattern alignment
Tensor Core utilization	MXU utilization
`__syncthreads()`	Implicit tile synchronization

DMA/compute overlap: the pipeline timeline. On both platforms, the highest-performance kernels overlap data movement with computation. While the arithmetic units process tile N, the memory system loads tile $N{+}1$. Throughput per tile is max(compute time, load time), not their sum.



DMA/compute/store pipeline. While tile N is computed, tile $N+1$ is loaded and tile $N-1$ is stored. On a GPU, this is double-buffering in shared memory. On a TPU, this is Pallas's `Buffered` *prefetching. The optimization thinking is identical; only the API differs.*

Memory hierarchy mapping.



Key message. The optimization *thinking* is identical across platforms: diagnose the bottleneck with the roofline, optimize data movement to reduce bytes transferred across the memory wall, overlap compute and memory access to hide latency. Only the API changes. A practitioner who understands the principles on one platform can transfer to another in days, not months. The roofline model is the portable skill.

Exercise: Cross-Platform Attention Optimization

Take the FlashAttention optimization from Move 2a (tile QKV computation to fit in SRAM, use online softmax to avoid materializing the N^2 matrix) and describe how you would implement the same optimization on a TPU using Pallas. Focus on the conceptual mapping, not the syntax.

Address these questions:

- FlashAttention tiles Q into blocks that fit in shared memory. On a TPU, what determines the tile size? (VMEM capacity and MXU dimensions.)
- FlashAttention uses `__syncthreads()` between loading a KV tile and computing attention scores. What is the TPU equivalent? (Implicit in Pallas's grid execution model, where each tile is a separate grid cell.)
- FlashAttention double-buffers KV tiles: loading tile $k+1$ while computing on tile k. How does this map to Pallas? (`Buffered(buffer_count=2)` in the `BlockSpec`.)
- The online softmax maintains running (m, ℓ) state across tiles. On a TPU, where does this state live? (VMEM scratch space via `scratch_shapes`, or VREGs if small enough.)
- FlashAttention's backward pass recomputes the forward attention from saved Q, K, V instead of storing the N^2 matrix. Does this recomputation strategy change on a TPU? (No, the memory-vs-recompute tradeoff is hardware-independent; the TPU's larger VMEM may

allow slightly larger tiles but the algorithm is the same.)

Write a one-paragraph summary: what is shared between the GPU and TPU implementations, and what must change?

Rollout Generation at Scale

Why This Matters at Frontier Scale

Dario Amodei put it plainly: "This isn't a research problem. This is an engineering and inference problem." RLVR training runs last weeks, and 50–70% of that wall time is spent on autoregressive token generation, not on gradient computation or optimizer updates, but on the forward pass that produces rollouts. A 2× speedup in generation throughput cuts your training time nearly in half.

Chapter 1 gave you the memory wall. Chapter 2 gave you the roofline model and kernel-level diagnosis. Now you compose kernels into a system: the rollout generation pipeline that feeds the RL training loop. The optimization target is not latency to a user. It is throughput (total tokens per second per dollar) under constraints that do not exist in user-facing serving. This distinction changes every design decision in the stack.

Move 1: Rollout Generation Is Not User-Facing Serving

User-facing inference optimizes **latency**: time-to-first-token (TTFT), time-per-output-token (TPOT), tail latency at P99. The workload is unpredictable: diverse queries, variable context lengths, bursty traffic patterns. The system must handle any prompt at any time.

RL rollout generation optimizes **throughput**: total tokens generated per second per dollar. The workload is predictable: every query runs against the same model (the current policy), prompts come from a known training distribution, and no human is waiting for the response. Latency of any individual rollout is irrelevant; what matters is the rate at which *batches* of rollouts complete so the training step can proceed.

This changes the design calculus at every level:

Design decision	User-facing serving	RL rollout generation
Batch size	Small (low latency)	As large as memory allows
Scheduling	First-come, first-served	Co-designed with training loop
Hardware target	Latency-optimized	Throughput-per-dollar
Model diversity	Many models, hot-swap	Single policy, fixed architecture
Quantization	Aggressive (latency)	Calibrated (signal quality)
Failure handling	Retry immediately	Drop straggler, resample

The single-model constraint is the most exploitable. In user-facing serving, the system must handle arbitrary model switches, adapter loading, and heterogeneous request shapes. In rollout generation, every request targets the same weights, uses the same vocabulary, and draws from the same prompt distribution. This means you can:

- **Pre-compute shared KV-caches** for common prompt prefixes and reuse them across all rollouts.
- **Tune batch sizes and tile configurations** offline for the specific model architecture, rather than using dynamic heuristics.
- **Fuse scheduling with the training loop**, overlapping generation of batch $N+1$ with gradient computation on batch N.

Think First (no computer): Rollout-Specific Design

List three specific design decisions in an inference serving system that would change if you knew all queries came from the same model, used the same prompt distribution, and no human was waiting for the response.
For each decision, state: (a) what the user-facing system does, (b) what the rollout system should do instead, and (c) what metric improves. Be concrete; cite numbers from Chapters 1 and 2 where relevant (e.g., arithmetic intensity shifts from batching, bandwidth savings from prefix sharing).

Move 2: The Prefill/Decode Split and On-Policy Freshness

Chapter 1 established that prefill and decode sit on opposite sides of the roofline: prefill is compute-bound (high arithmetic intensity, processing all prompt tokens in parallel), decode is memory-bandwidth-bound (low arithmetic intensity, generating one token at a time). This split creates a systems design opportunity.

Disaggregated serving. Instead of running prefill and decode on the same hardware with the same configuration, split them: run prefill on compute-optimized hardware (or with large tile sizes and high utilization of the MXU/Tensor Cores), and decode on bandwidth-optimized hardware (or with configurations tuned for memory throughput). In user-facing serving, this disaggregation is useful but complex to manage because workloads are unpredictable. In rollout generation, *you control the workload*. You know exactly how many prompts arrive, their length distribution, and the expected output length. You can tune the prefill-to-decode hardware ratio precisely, statically, based on profiling data from the training distribution.

The prefill/decode arithmetic. Consider a 7B model processing prompts of $P = 2048$ tokens and generating rollouts of $D = 1024$ tokens at batch size $B = 64$:

- **Prefill cost**: One large batched matmul. FLOPs $\approx 2 \times P \times B \times \text{params} = 2 \times 2048 \times 64 \times 7\text{B} \approx 1.8 \times 10^{15}$ FLOPs. At 500 TFLOP/s (H100), this takes ≈ 3.6 seconds. High utilization.
- **Decode cost**: $D = 1024$ sequential steps, each a batched matrix-vector multiply. At batch size 64, decode is still memory-bound (Chapter 2: arithmetic intensity ≈ 64 FLOP/byte on H100 with ridge point 148). Total wall time dominated by bandwidth: loading 14 GB of weights (BF16) per step at 3.35 TB/s takes ≈ 4.2 ms/step, so $1024 \times 4.2 \approx 4.3$ seconds.

Prefill and decode take comparable time in this regime, but use hardware differently. Running both on the same GPUs means the hardware is alternatively over-provisioned in compute (during decode) and in bandwidth (during prefill). Splitting them lets each phase use its hardware efficiently.

On-policy freshness: the deadline that does not exist in serving. RL rollout generation has a constraint that user-facing serving lacks entirely: **on-policy freshness**. The RL algorithm (GRPO, PPO, REINFORCE) computes policy gradients from rollouts sampled under the *current* policy π_θ. As training proceeds, the policy updates: $\theta \to \theta'$. Rollouts generated under the old policy π_θ become stale, and the gradient computed from them is biased relative to the current policy.

Off-policy methods (PPO with importance sampling, for instance) can tolerate some staleness by reweighting, but the importance weights $\pi_{\theta'}(a|s)/\pi_\theta(a|s)$ increase in variance as the policies diverge. GRPO, which is on-policy, has zero tolerance for staleness: rollouts from the wrong policy produce incorrect advantage estimates.

This creates a **deadline**: rollouts must be generated fast enough that the policy has not significantly

changed by the time the gradient step uses them. The tradeoff:

- **Larger batches** amortize memory access costs, increasing throughput. But larger batches take longer to generate, increasing staleness.
- **Smaller batches** complete faster, maintaining freshness. But smaller batches waste bandwidth (lower amortization of weight loads) and produce noisier advantage estimates.

The sweet spot depends on the policy's rate of change, which itself depends on the learning rate, reward signal magnitude, and task difficulty. There is no universal answer. The skill is knowing this tradeoff exists and measuring it for your specific setup.

Move 3: Actor-Learner Architectures

RLVR training is not a single workload. It is a pipeline of three distinct compute phases that must be orchestrated across the same or overlapping hardware:

1. **Rollout generation (inference).** The policy generates G completions per prompt (GRPO uses $G \sim 8$–16). This is memory-bandwidth-bound (Chapter 1). GPU utilization during generation: typically 30–60%, even with batching.
2. **Reward evaluation.** Each completion is scored by a reward model or verifier. This is a separate forward pass. If the reward model differs in architecture, it needs its own GPU partition or must time-share.
3. **Policy update (training).** Advantages are computed, and a gradient step is taken. This is compute-bound: large matrix multiplications in the backward pass. GPU utilization should be high, provided the batch is large enough.

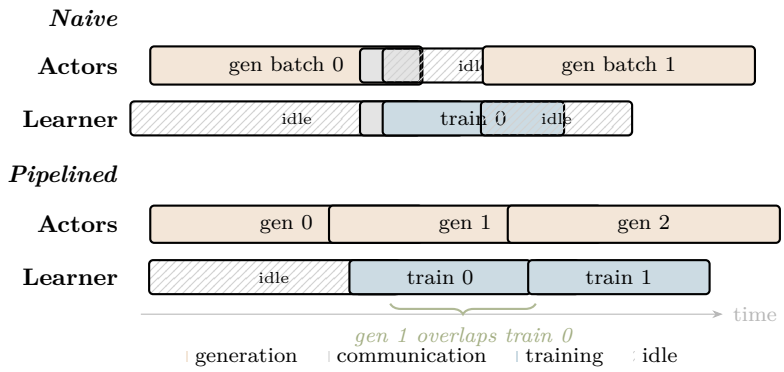
The actor-learner architecture separates these phases onto dedicated hardware:

- **Actors**: nodes dedicated to rollout generation. Optimized for inference throughput: large batches, quantized weights, continuous batching.
- **Learners**: nodes dedicated to gradient computation and weight updates. Optimized for training throughput: full precision, FSDP or tensor parallelism, large effective batch size.

The ratio question: how many actors per learner? This is a capacity planning problem with a feedback loop. If actors outpace learners, rollouts queue up and go stale before the learner can consume them, violating the on-policy freshness constraint from Move 2. If learners outpace actors, learners starve: expensive training hardware sits idle waiting for rollouts. The optimal ratio depends on the generation-to-training time ratio, which depends on model size, sequence length, G, and hardware.

Variable-length outputs and the straggler problem. Completions vary in length. A prompt that terminates early leaves its actor idle while others finish. The training step cannot begin until *all* G completions per prompt are done (needed for advantage normalization in GRPO). The slowest completion in the batch determines when training can start. As the number of actors N grows, the probability that at least one has a slow completion increases, and the expected straggler delay grows as $O(\log N)$ for i.i.d. completion times.

Pipelining: overlap generation and training. The key optimization is to never let the learner wait. While the learner trains on batch N, actors generate batch $N + 1$. This hides the generation latency behind training compute, provided generation is faster than or equal to training. When generation is the bottleneck (the common case, 50–70% of wall time), pipelining reduces but does not eliminate the bubble.



Actor-learner pipeline: naive vs. pipelined execution. In the naive version, actors idle during training and the learner idles during generation. Pipelining overlaps generation of batch $N+1$ with training on batch N. The initial idle bubble (learner waiting for the first batch) is unavoidable; subsequent bubbles shrink to the difference between generation and training time.

Weight synchronization. After the learner completes a gradient step, updated weights must be broadcast to all actors. This communication cost grows with model size and actor count. For a 70B model in BF16, the weight tensor is 140 GB. Broadcasting to A actors over InfiniBand (400 Gb/s = 50 GB/s) takes $140/50 \approx 2.8$ seconds per actor (sequentially) or ≈ 2.8 seconds total with a tree broadcast. This is dead time in which neither generation nor training is happening. Overlapping weight broadcast with the end of generation (actors finish their current rollouts while receiving new weights) is essential at scale.

Design heuristic. Start with a 1:1 actor-to-learner ratio. Profile. If actors are the bottleneck (learner idle time $> 10\%$), add actors. If the learner is the bottleneck, reduce actors or increase the training batch size. The goal is to keep both sides saturated with zero idle time, which in practice means keeping idle time below 5%.

Move 4: The Optimization Toolkit

With the system architecture established (actors generating rollouts, learners computing gradients, a pipeline connecting them), we now equip each component with the optimizations that matter for throughput. Each technique below was invented for user-facing serving but takes on different characteristics when applied to RL rollout generation.

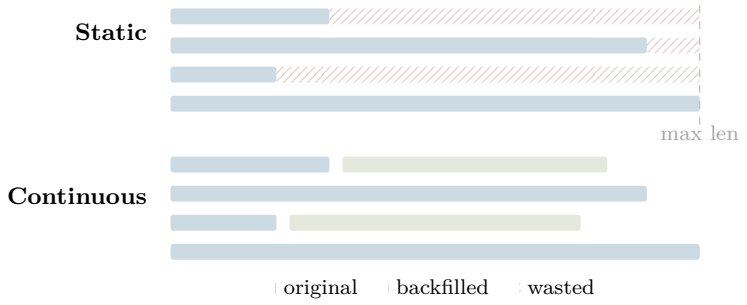
Continuous Batching

RLVR rollout generation produces extreme length variance. A single GRPO step generates G completions per prompt (typically $G = 8$–32) for each of B prompts, producing $G \times B$ concurrent sequences. For reasoning models, sequence lengths follow a heavy-tailed distribution: many completions finish quickly (100–300 tokens for easy problems), while hard problems produce extended chains of thought (1000–4000 tokens). The ratio $\ell_{\max}/\ell_{\text{mean}}$ can exceed 10:1.

Static batching wastes compute. The naive approach pads all sequences to the maximum length in the batch. For a batch where 90% of sequences are 200 tokens and 10% are 2000 tokens, static batching computes $10\times$ as many tokens as needed for the short sequences, and 90% of GPU cycles in the tail are wasted on padding.

Continuous batching. As one sequence finishes, a new one enters the batch immediately. The GPU stays busy with real work instead of padding. This is the core technique in vLLM, TGI, and

similar serving systems.



Static vs. continuous batching. Static batching pads all sequences to the maximum length (hatched area = wasted compute). Continuous batching inserts new sequences (green) as old ones complete, keeping the GPU fully utilized.

The GRPO synchronization constraint. Continuous batching across independent requests is straightforward. RLVR adds a constraint: all G completions for a single prompt must complete before advantage computation can proceed. The advantage $\hat{A}_i$ for completion i is computed relative to the group:

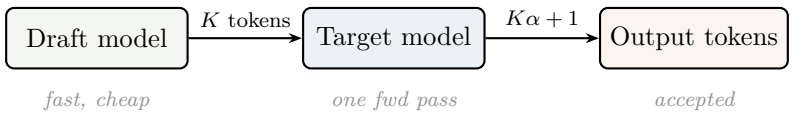
$$\hat{A}_i = \frac{r_i - \mu_G}{\sigma_G}$$

where μ_G and σ_G are the mean and standard deviation of rewards within the group of G completions. You cannot compute μ_G until all G completions are finished. This creates a *prompt group* abstraction: G sequences that must be synchronized before training can begin.

Group-aware continuous batching. The solution: batch *across* prompt groups while maintaining per-group synchronization. When a completion finishes, its slot is immediately filled by a completion from a new prompt group. When all G completions in a group finish, that group's advantages are computed and the group enters the training queue. Long-running groups coexist with short groups in the same batch. The key insight: you do not wait for all groups to finish, only for each individual group. This gives continuous batching's throughput benefit while respecting GRPO's synchronization requirement.

Speculative Decoding

A small *draft model* proposes K tokens; a single forward pass of the large *target model* verifies all K at once. This converts sequential decode into partially parallel work.



The expected number of tokens produced per target forward pass is $K\alpha + 1$, where α is the per-token acceptance rate. Standard autoregressive produces exactly 1 token per pass. Let the draft model cost c_d per token and the target cost c_t per token. Speculative decoding wins when:

$$Kc_d + c_t < (K\alpha + 1)c_t \implies \frac{c_d}{c_t} < \alpha$$

The breakeven condition: the acceptance rate α must exceed the cost ratio c_d/c_t. If the draft model is 10× cheaper, speculative decoding helps for $\alpha > 0.1$.

RL-specific advantage: the draft model is free. In user-facing serving, you need a separate distilled draft model. In RL training, the previous policy checkpoint is a natural draft model that already exists, requires no distillation, and approximates the current policy. The acceptance rate α depends on how much the policy shifted in k gradient steps. Early in training (small updates), α is high and speculative decoding provides large speedups. Late in training (policy converging), α remains high because updates are small. The dangerous period is mid-training, where the policy changes rapidly.

The moving-target problem. As the policy updates, the draft model (previous checkpoint) diverges from the target (current policy). Acceptance rate degrades monotonically with divergence. At some training step t^*, α falls below breakeven and speculative decoding stops helping. The practical question: how often must you refresh the draft checkpoint? This depends on learning rate, reward magnitude, and task difficulty. Measure α periodically during training. When it drops below $2 \times \alpha^*$ (a safety margin), refresh the draft.

KV-Cache Prefix Sharing

In RL rollout generation, many rollouts share the same task prompt. GRPO generates G completions per prompt, all starting from the same prefix. This is a free optimization that does not exist in user-facing serving with diverse queries.

Compute the prompt KV-cache once. Run prefill on the shared prompt, producing the KV-cache for all L layers. Store this cache. For each of the G rollouts, begin autoregressive decode from this cached state. The prefill cost is amortized $G\times$: instead of G prefills, you do 1.

Memory cost. The shared prefix cache must be stored for the duration of the group's generation. For a 70B model with 80 layers, 64 heads, head dim 128, at BF16 and prompt length $P = 2048$:

$$\text{Prefix cache} = 2 \times 80 \times 64 \times 128 \times 2048 \times 2 \text{ bytes} \approx 5.4 \text{ GB}$$

This is a one-time allocation per prompt group, versus $5.4 \times G$ GB without sharing. At $G = 16$, prefix sharing saves ≈ 81 GB of KV-cache memory, more than an entire H100's HBM.

Implementation. vLLM's prefix caching and SGLang's RadixAttention both support this pattern. The system detects common prefixes across requests and maps them to the same physical KV-cache blocks. For RL workloads where the sharing pattern is known in advance (same prompt, G completions), the scheduler can be even simpler: explicitly share the prefix and fork G decode streams.

Quantization at System Level

Chapter 2 analyzed quantization kernel-by-kernel: the roofline shift from FP16 to INT4 doubles arithmetic intensity, moving decode closer to the compute-bound regime. Here the question is end-to-end: **does quantization improve policy improvement per dollar?**

The mechanism is clear: quantized weights reduce bytes loaded per decode step, increasing throughput (tokens/second). The cost is generation quality degradation from quantization noise. In su-

pervised serving, this tradeoff is straightforward: measure perplexity, accept the degradation if it is small. In RL, the stakes are different.

The policy gradient is $\nabla_\theta \log \pi_\theta(a|s) \cdot \hat{A}$, where $\hat{A}$ is the advantage. Policy gradient estimates are inherently high-variance. Quantization noise adds an additional perturbation on top of this already-noisy estimator. The concern is that the failure mode is silent: training proceeds, rewards increase, but at a slower rate than full precision would achieve. Without a baseline to compare against, you would not know.

The measurement. Run the same training setup at FP16 and INT4 (or INT8) for a fixed wall-clock budget (not a fixed number of steps). INT4 generates more rollouts per hour (higher throughput), but each rollout is slightly lower quality. The question: does the throughput gain outweigh the quality loss? Plot reward versus wall-clock time for both configurations. If the INT4 curve stays above the FP16 curve, quantization is a net win even though individual rollouts are noisier. If it falls below, the quality loss dominates. The crossover point depends on model size, task difficulty, and quantization method. Measure before committing to a weeks-long run.

MoE Serving for RL Workloads

Mixture-of-Experts models replace the dense FFN with E parallel expert FFNs, routing each token to the top-k experts. In distributed serving, experts are sharded across GPUs (expert parallelism). Each token's computation requires all-to-all communication: send token representations to the GPU holding the selected expert, compute, send results back. For a model with $L/2$ MoE layers (alternating dense and MoE), that is L all-to-all operations per autoregressive step.

All-to-all scales poorly. All-to-all volume per device is $O(N)$ where N is the number of GPUs (each device sends a different chunk to every other device). Compare this to ring all-reduce, which is $O(S)$ per device regardless of N. There exists a critical N^* beyond which adding more GPUs increases communication time faster than it adds parallel throughput. Identifying N^* for your MoE deployment is essential before scaling up.

Load imbalance from narrow RL distributions. In pretraining, the training distribution is broad (internet-scale text), and expert routing distributions are approximately stationary. In RL training on a specific task (math, code, reasoning), the prompt distribution is narrow. Some experts become disproportionately popular for the task, while others are underutilized. This creates dynamic load imbalance: GPUs holding hot experts are saturated while cold-expert GPUs are idle. The load-balancing auxiliary loss corrects for this over multiple gradient steps, but during the transition period, throughput drops.

Expert caching. For a fixed prompt distribution, a small number of hot experts handle most tokens. Keeping hot experts in fast memory (HBM or L2) and evicting cold experts to CPU DRAM reduces effective all-to-all volume. Under RL training, the hot set shifts as the policy changes, so caching must adapt dynamically.

Tensions

Three tensions run through this chapter. Each has no universal answer; the right point depends on your specific model, task, and hardware. The skill is knowing where the tension exists and measuring where the boundary falls.

Tension 1: Batch size vs. on-policy freshness.

Larger batches amortize memory access costs. At batch size B, the weight-loading cost per token during decode is params $\times\, 2/B$ bytes, versus the full weight load at $B = 1$. Doubling B halves the

per-token bandwidth cost. But larger batches take longer to generate, and the policy may have updated by the time the batch finishes, making the rollouts stale.

The boundary: at what batch size does the staleness penalty (biased gradients, increased variance from importance-weight correction) outweigh the throughput benefit?

Exercise: Finding the Freshness Boundary

Design an experiment to find the batch-size/freshness tradeoff boundary. Use a small model (1B–3B parameters) on a single GPU with a math-reasoning task (GSM8K or similar).

1. Fix the total number of tokens generated per training step (e.g., $N = 32{,}768$ tokens).
2. Vary batch size $B \in \{32, 64, 128, 256, 512\}$. For each B, the number of prompts is $N/(B \times \text{avg\_length})$.
3. For each B, measure: (a) wall-clock time per training step, (b) tokens per second, (c) reward improvement after 200 gradient steps.
4. Plot throughput (tokens/s) vs. B and reward vs. B. The throughput curve should increase with B (bandwidth amortization). The reward curve should peak and then decline (staleness).
5. Identify the B^* that maximizes reward improvement per wall-clock hour.

What is the shape of the reward-vs-batch-size curve? Is the decline gradual or sharp?

Tension 2: Quantization vs. signal quality.

Quantization cuts the cost per rollout by reducing bytes loaded per decode step. It also degrades generation quality, introducing noise into the reward signal and therefore into the gradient.

The boundary: at what quantization level does the gradient signal-to-noise ratio degrade enough that policy improvement per dollar decreases despite higher throughput?

Exercise: Quantization Signal-to-Noise

Design an experiment to measure where quantization degrades RL signal quality.

1. Take a pre-trained model (3B–7B parameters) and a verifiable task (math problems with ground-truth answers).
2. Run GRPO training at three precision levels: BF16, INT8, INT4. Use the same hyperparameters for all three.
3. For each precision level, log: (a) tokens generated per second, (b) reward variance within each GRPO group, (c) reward mean at each training step.
4. Compute the effective metric: reward improvement per GPU-hour for each precision level.
5. If INT4 generates 2× the tokens/second but achieves 0.7× the reward improvement per step, the net efficiency is 1.4×, still a win. If it achieves 0.4× per step, the net is 0.8×, a loss.

At what precision does the curve cross? Does the answer change between easy tasks (binary reward) and hard tasks (long chain-of-thought)?

Tension 3: Actor/learner ratio.

Over-provisioning actors wastes compute: excess rollouts go stale before the learner can process them. Under-provisioning starves the learner: expensive training hardware sits idle.

The boundary: what is the optimal ratio, and how does it shift as training progresses (generation length changes, policy update time changes)?

Exercise: Actor/Learner Ratio Sweep

Design a simulation (no GPU required) to find the optimal actor-to-learner ratio.

1. Model the system: actors generate rollouts at rate R_a tokens/second each. The learner consumes rollouts at rate R_l tokens/second. Weight synchronization takes T_{sync} seconds per update.
2. For A actors, total generation rate is $A \times R_a$. The learner processes at rate R_l. Steady state requires $A \times R_a \approx R_l$.
3. Simulate 100 training steps with $A \in \{1, 2, 4, 8, 16\}$ actors, using realistic values: $R_a = 5000$ tokens/second (per actor), $R_l = 50{,}000$ tokens/second (learner), $T_{\text{sync}} = 3$ seconds, generation length sampled from log-normal($\mu = 6$, $\sigma = 1$).
4. For each A, measure: (a) learner utilization (fraction of time computing gradients), (b) average rollout staleness (seconds between generation and training), (c) total training steps completed in one hour.
5. Plot all three metrics vs. A. Identify the A^* that maximizes training steps per hour while keeping average staleness below a threshold you choose.

How sensitive is A^* to the staleness threshold? What happens when generation length variance doubles?

Portfolio Project

Portfolio Project: Rollout Generation Profiler

The question. What fraction of RLVR training compute is wasted on generation overhead, and which optimization provides the largest improvement per engineering hour?

The riskiest assumption. That you can identify and eliminate the dominant generation bottleneck without a full-scale training run. Profile a real RLVR training loop, identify the largest throughput bottleneck in rollout generation, and implement one optimization that measurably improves tokens-per-second-per-dollar.

The minimal viable setup. A profiling harness that instruments a GRPO training loop (minimum 1B-parameter policy, real reward signal) and produces per-step breakdowns of: prefill time, decode time, KV-cache overhead, synchronization wait time (straggler bubble), weight transfer time, and training time.

Implementation plan.

1. Run the RLVR loop uninstrumented for 100 steps to establish baseline step time and throughput.
2. Add `torch.profiler` instrumentation. Run 20 steps. Export traces. Identify the top-3 time sinks per step.
3. For the largest generation-side bottleneck (>10% of step time), implement one optimization from this chapter: continuous batching, KV-cache prefix sharing, speculative decoding with a previous checkpoint, or quantized generation.
4. Measure the improvement. Compare: step time, tokens/second, and useful compute fraction ($\eta = t_{\text{train}}/t_{\text{step}}$).
5. Write the blog post with before/after profiler traces, a utilization breakdown chart, and one concrete recommendation for practitioners.

Verification criteria.

1. Profiler trace shows at least one generation-side bottleneck >10% of step time.
2. Implemented optimization reduces that bottleneck by >20% in measured wall time.
3. Blog post contains reproducible measurements and a concrete takeaway an RL team could act on.

Policy Gradients

Why This Matters at Frontier Scale

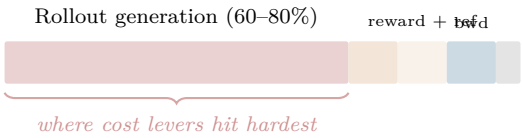
You built a rollout generation system (Chapter 3). This chapter teaches you what happens to those rollouts, and why understanding the consumer is essential to optimizing the producer. Every algorithmic choice here has a systems consequence: it changes how many rollouts you need, how fresh they must be, and how much each one costs.

The RL Loop as Consumer of Rollouts

Policy gradient methods use rollouts to compute weight updates. The training loop is the consumer; the generation pipeline from Chapter 3 is the producer. The quality, diversity, and freshness of rollouts directly determine whether each update improves the policy or wastes compute.

One GRPO training step breaks down as:

```
wall_clock_per_step =
    rollout_generation    (num_prompts * G * seq_len / throughput)
       60-80%
  + reward_computation    (num_prompts * G * verifier_cost)
     5-10%
  + reference_forward     (num_prompts * G * ref_model_fwd)
     5-10%
  + backward_pass         (batch_size * model_size)
     5-15%
  + gradient_comm         (gradient_size / interconnect_bw)
     2-5%
```



Ranges are regime-dependent and shift with model size, sequence length, and hardware. The point is the ordering, not the exact numbers. Rollout generation is 60–80% of wall-clock time, which is why Chapter 3 is the centerpiece of this book. Cutting rollout count in half saves 30–40% of total training cost. Cutting optimizer steps in half saves 3–8%.

Your job as you read this chapter: understand the consumer well enough to make informed producer decisions. The batch size, quantization strategy, and actor/learner ratio you chose in Chapter 3 for throughput reasons all have direct algorithmic consequences here. Each lever below tells you what those consequences are.

Think First (no computer): Cost Asymmetry

Two estimation problems. Work both on paper before reading the next section.
(a) Your GRPO pipeline runs $G = 16$ rollouts per prompt. You discover a variance reduction technique that gives equivalent gradient signal quality at $G = 8$. Using the cost breakdown above, estimate the wall-clock savings as a percentage of total step time.
(b) You discover a second-order optimizer that converges in half the steps. Same question:

estimate the wall-clock savings.
Which improvement would you staff an engineer on first?

The four levers, ordered by cost impact:

1. **Rollout allocation:** how many rollouts per prompt, and which prompts get them
2. **KL regularization:** how tightly the policy tracks its reference
3. **Credit assignment:** whether gradient signal targets the tokens that mattered
4. **Collapse detection:** how early you detect that continued training is waste

The first three are ordered by steady-state cost impact. The fourth prevents catastrophic waste rather than improving per-step efficiency, since a single early intervention can save more compute than all other levers combined.

Lever 1: Rollout Allocation and Clipping

Think First (no computer): Optimal Group Size

The GRPO gradient estimator for a single prompt with G rollouts is:

$$\hat{g} = \frac{1}{G}\sum_{i=1}^{G} \nabla_\theta \log \pi_\theta(\tau_i) \cdot \hat{A}_i, \quad \hat{A}_i = \frac{R_i - \bar{R}}{\sigma_R}$$

The variance of this estimator scales as $\text{Var}(\hat{g}) \propto \sigma_A^2/G$. Doubling G halves variance but doubles the dominant cost term.
Derive: the cost-optimal G minimizes total compute to reach a target reward. If convergence requires $N(G)$ steps and each step costs $c_{\text{base}}+c_{\text{rollout}}\cdot G$, the total cost is $N(G)\cdot(c_{\text{base}}+c_{\text{rollout}}\cdot G)$. Under the simplifying assumption $N(G) \propto 1/G$ (linear variance reduction), cost is minimized as $G \to \infty$. Why does this not hold in practice? What determines the finite optimum?

The assumption $N(G) \propto 1/G$ breaks because $N(G)$ flattens once gradient variance drops below the *bias floor*, the irreducible error from advantage estimation, stale rollouts, and reward noise. Below that floor, additional rollouts reduce variance that was not the binding constraint. The real optimum is finite and depends on the prompt distribution.

Waste pattern: dead rollouts. GRPO normalizes advantages within each group: $\hat{A}_i = (R_i - \bar{R})/\sigma_R$. For prompts where all G rollouts get similar rewards ($\sigma_R \approx 0$), normalized advantages are near-zero. Those rollouts produce negligible gradient signal. If 30% of prompts have low reward variance (easy problems the model already solves reliably, or hard problems it consistently fails), then ∼30% of the rollout budget produces near-zero learning.

Cost impact. Napkin math: a 7B model GRPO run at $G = 16$ with 100K prompts over 5K steps generates 8 billion rollout tokens. If 30% are dead rollouts, that is 2.4 billion tokens of inference for near-zero gradient signal. At ∼\$0.01 per 1K inference tokens, that is ∼\$24K wasted on a single run.

Systems reality. G rollouts from the same prompt share the prompt's KV cache, so the prompt forward pass is computed once and reused across all G completions. Going from $G = 8$ to $G = 16$ adds 8 completion forward passes, not 8 full forward passes. This makes larger G cheaper than the naive cost model predicts.

But: KV cache for G completions must fit in HBM simultaneously. At $G = 16$ with long prompts,

you can hit memory limits that force smaller batch sizes, which hurts GPU utilization. The real constraint is $G \times \text{batch\_size} \times \text{seq\_len} \leq \text{HBM budget}$.

Batch size interacts with clipping. Too-small batches produce high-variance advantage estimates that clipping cannot save. PPO/GRPO clipping bounds the importance ratio $\rho_t = \pi_\theta(a_t|s_t)/\pi_{\text{old}}(a_t|s_t)$ to $[1-\epsilon, 1+\epsilon]$. This limits the size of any single update, but does nothing about the *direction*. If the advantage estimate is wrong due to high variance, clipping faithfully executes a wrong update at a bounded magnitude. The batch size you chose in Chapter 3 for throughput reasons has a direct algorithmic consequence here: it determines whether advantage estimates are reliable enough for clipping to help rather than merely limit damage.

Literature (established solutions). This problem is solved. Multiple papers from early 2026 address adaptive rollout allocation:

- **VIP** (arXiv:2602.01601, 2026): Variance-Informed Predictive allocation. A Gaussian process predicts per-prompt success probabilities from recent rollouts. These predictions feed a convex optimization problem that allocates rollout budget to minimize expected gradient variance under a hard compute constraint.
- **AERO** (arXiv:2602.14338, 2026): Adaptive rollout strategy with selective rejection and a Bayesian posterior to prevent zero-advantage dead zones. Reports 48% compute reduction on 1.5B and 47% on 7B models.
- **Prompt Replay** (arXiv:2603.21177, 2026): On-policy reuse of high-signal prompts by skipping prompts that have become too easy or too hard, replay prompts in the high-learning zone.

Exercise: Rollout Allocation Efficiency

Part 1: Fixed G sweep. Train GRPO on TinyRL-Code (Chapter 5, 124M config) with $G \in \{4, 8, 16, 32\}$. For each G, measure:

- Reward per GPU-second (primary metric)
- Dead rollout rate: fraction of prompts per step where $\sigma_R < 0.1$
- Gradient norm variance across steps

Plot reward/GPU-second vs. G. Is there a clear optimum?

Part 2: Adaptive allocation. Implement a simplified VIP-style scheme: maintain a rolling window of per-prompt reward variance. Each step, allocate $G_i \propto \hat{\sigma}_{R,i}$ under a fixed total rollout budget equal to $G = 8 \times \text{num\_prompts}$. Compare reward/GPU-second against the best fixed G from Part 1.

Part 3: Analysis. Does the dead rollout rate increase over training (as the policy improves on easy prompts)? If so, a fixed G becomes increasingly wasteful. Plot dead rollout rate over training steps for each fixed G. Does adaptive allocation keep the dead rollout rate low?

What to investigate next. Read VIP and AERO in full. Both assume reward variance is a good proxy for gradient information, but is this true when the advantage estimator introduces its own bias? Compare VIP's Gaussian process approach against AERO's Bayesian posterior. Which is cheaper to maintain at scale?

Lever 2: KL Regularization

Think First (no computer): The KL Penalty as Resource Allocation

GRPO adds a KL penalty to the reward:

$$R'(\tau) = R(\tau) - \beta \cdot D_{\mathrm{KL}}(\pi_\theta \| \pi_{\mathrm{ref}})$$

This is equivalent to optimizing a variational lower bound:

$$\max_\theta \ \mathbb{E}_{\pi_\theta}[R(\tau)] - \beta \cdot D_{\mathrm{KL}}(\pi_\theta \| \pi_{\mathrm{ref}})$$

The coefficient β controls how far the policy drifts from the reference. A fixed β applies the same exploration-stability trade-off for the entire run.
Think through the consequences at two extremes of training:

1. **Early training:** the policy is far from any reward-hacking regime and the gradient signal is noisy. Does a fixed β help or hurt?
2. **Late training:** the policy is near a good solution. Small perturbations can trigger entropy collapse. Does the same β help or hurt?

What would an adaptive schedule look like?

Waste pattern. Over-regularization wastes gradually: each step is slightly less efficient because β constrains updates that would have been safe. Under-regularization wastes catastrophically: the policy drifts into collapse or reward-hacking, and hundreds of steps are spent recovering. The cost of getting β wrong is dominated by the catastrophic tail.

Early in training, β barely constrains, and the reference forward pass (5–10% of step time per the cost breakdown) produces no useful signal. Late in training, the same β may be too loose, causing collapse-and-recovery episodes that consume hundreds of steps.

The freshness connection. KL penalty directly determines how quickly your rollouts go stale. Tight KL (large β) means the policy moves slowly, so rollouts stay on-policy longer, relaxing Chapter 3's freshness constraint. You can get away with a larger actor/learner ratio and more reuse of generated rollouts. Loose KL (small β) means the policy changes fast, so you need fresher rollouts, more generation throughput, and tighter synchronization between the actor and learner from Chapter 3. The algorithmic choice of β has a direct systems consequence for how you architect the generation pipeline.

Systems reality. The reference model consumes HBM. Common workarounds and their cost profiles:

- **CPU offload:** frees HBM but adds latency for each KL computation.
- **Weight sharing with LoRA:** reference = frozen base, actor = base + adapter. Eliminates the duplicate model but constrains the policy to a low-rank offset.
- **Asynchronous KL:** compute KL on a separate device or with a stale snapshot. Reduces latency on the critical path but introduces approximation error.

Each workaround changes the cost of evaluating KL, which changes when adaptive scheduling breaks even.

Literature:

- "Rethinking KL Regularization in RLHF" (arXiv:2510.01555, 2025): argues GRPO's KL im-

plementation overlooks its functional role as optimization loss.

- **SAFE** (arXiv:2602.04651, 2026): PID-controlled adaptive KL thresholds with entropy-gated regulation. Formalizes adaptive β using control theory.
- **APO** (arXiv:2512.22953, 2025): α-divergence interpolation between forward and reverse KL with confidence-guarded scheduling.

Exercise: KL Scheduling Comparison

Train GRPO on TinyRL-Code (Chapter 5, 124M) under four conditions:

(a) Fixed $\beta = 0.01$
(b) Fixed $\beta = 0.1$
(c) Simple adaptive: multiply β by 1.5 when $D_{\text{KL}} > 0.05$, by 0.67 when $D_{\text{KL}} < 0.01$
(d) PID-controlled β (reproduce SAFE's approach): target $D_{\text{KL}} = 0.03$, proportional gain $K_p = 1.0$, integral gain $K_i = 0.1$, derivative gain $K_d = 0.01$

Measure at every step: reward/GPU-second, $D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}})$, policy entropy. Count entropy collapse events (entropy < 0.1 for > 10 consecutive steps).

Questions. Does PID outperform the simple heuristic? By enough to justify the implementation complexity? Does adaptive scheduling prevent collapse episodes that the fixed conditions hit? Which fixed β is closer to the adaptive performance?

What to investigate next. Read SAFE and APO. SAFE uses PID control; is the integral term necessary, or does proportional control suffice? APO schedules across divergence families, not just β magnitude; does the choice of forward vs. reverse KL matter more than the coefficient?

Lever 3: Credit Assignment

Think First (no computer): Where Does the Gradient Signal Go?

With sequence-level rewards, the policy gradient applies the same advantage $\hat{A}$ to every token:

$$\hat{g} = \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot \hat{A}$$

In a 512-token math reasoning trace, suppose ∼20 tokens are critical (an algebraic manipulation, a key substitution). The other 492 tokens are setup or mechanical follow-through.

(a) What fraction of the gradient variance comes from non-critical tokens? (Each token contributes $\|\nabla_\theta \log \pi_\theta(a_t|s_t)\|^2 \cdot \hat{A}^2$ to the variance.)

(b) How does this connect to the rollout budget from Lever 1? If gradient variance is 25× higher than it needs to be, how many extra rollouts does that require to compensate?

The naive story that "GRPO only does sequence-level credit" is wrong. Recent work shows GRPO already performs implicit token-level credit assignment.

The implicit credit mechanism. Sullivan et al. (arXiv:2509.21154, "GRPO is Secretly a PRM," 2025) prove that GRPO with outcome rewards is equivalent to a process-reward-model-aware RL objective with Monte-Carlo-based process rewards, whenever trajectories within a group share prefixes. In practice, G rollouts from the same prompt often share long prefixes because the model generates similar openings. Where prefixes overlap, GRPO assigns differential credit to the divergence points: implicit token-level credit without a critic.

This implicit mechanism breaks down when:

- Prefixes are short (the model generates diverse openings, common for creative or open-ended tasks)
- G is small (fewer shared prefixes)
- The critical decision is early in the sequence (before prefixes diverge)

Waste pattern. When the implicit mechanism fails, you pay the full gradient variance tax from uniform credit. This inflates the required G for stable training, linking back to Lever 1. Poor credit assignment is a hidden multiplier on rollout cost.

GAE and rollout length. Generalized Advantage Estimation (GAE) uses an exponentially-weighted combination of n-step returns, controlled by λ. When λ is high, GAE propagates reward signal further back in the trajectory, giving better credit assignment for early decisions, but only if the rollout is long enough to contain the relevant signal. GAE's λ parameter interacts with rollout length: longer rollouts give GAE more signal to work with, but cost more to generate in Chapter 3. This creates a direct link between credit assignment quality and your generation throughput budget.

Explicit alternatives. Multiple methods improve on implicit credit, each with a different compute profile:

- **GRPO-λ** (OpenReview): eligibility traces using token-level log-probabilities. Explicit credit without a learned critic. Negligible overhead; reuses existing log-probs.
- **GTPO/GRPO-S** (arXiv:2508.04349, 2025): entropy-weighted per-token reward shaping via Dynamic Entropy Weighting. Small overhead (entropy computation per token).
- **SPO** (arXiv:2505.23564, 2025): segment-level advantages. More precise than sequence-level, fewer estimation points than token-level. Moderate overhead (segment boundary detection).
- **Learned critic** (PPO-style): ∼30–50% memory overhead + ∼20% step time for critic forward and backward passes.

The trade-off: explicit credit methods add per-step cost but reduce required G. Whether this nets positive depends on the ratio of rollout cost to credit-method cost.

Exercise: Credit Assignment Strategies

Train GRPO on TinyRL-Code (Chapter 5, 124M) with four credit assignment strategies:

(a) Standard GRPO (implicit credit via shared prefixes)
(b) GRPO-λ (eligibility traces)
(c) GTPO (entropy-weighted token rewards)
(d) Heuristic: assign zero advantage to tokens outside code blocks (task-structure proxy for "where the reasoning happens")

Measure: reward/GPU-second (including all overhead), gradient norm variance, convergence speed to 80% solve rate.

Key question. At what G does standard GRPO (a) match GRPO-λ's (b) performance at $G = 8$? The gap in G is the credit assignment tax in rollout budget.

Prefix analysis. Measure the actual prefix-sharing rate in your runs: what fraction of rollout pairs within a group share $> 50\%$ of tokens? If high, implicit credit may be sufficient and explicit methods add cost for little gain. If low, GRPO-λ or SPO become high-value.

What to investigate next. Read "GRPO is Secretly a PRM" carefully. The proof relies on shared prefixes. Does prefix-sharing rate change over training (as the policy becomes more deterministic)? If so, implicit credit improves as training progresses, exactly when it matters most.

Lever 4: Collapse Detection and Entropy

Entropy collapse is well-studied with specific mechanistic findings. This section teaches the established results; the experiment validates rather than discovers.

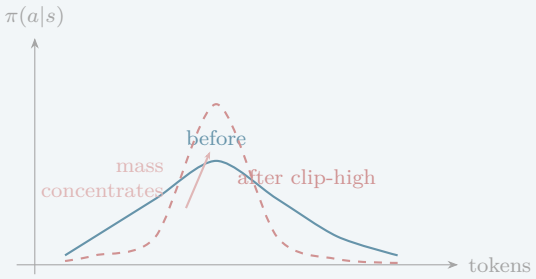
Think First (no computer): Why Does Clipping Cause Entropy Collapse?

Entropy collapse occurs when $H(\pi_\theta(\cdot|s)) \to 0$: the policy becomes near-deterministic. When this happens, $\nabla_\theta \log \pi_\theta(a|s)$ diverges for low-probability actions, creating extreme gradient variance. The policy is stuck.

The cause is structural, not just a symptom of reward overfitting. Consider PPO/GRPO clipping with the importance ratio $\rho_t = \pi_\theta(a_t|s_t)/\pi_{\text{old}}(a_t|s_t)$:

- **Clip-high** (capping ρ_t for negative advantages, $\hat{A}_t < 0$): prevents the policy from *decreasing* probabilities too fast. But this asymmetrically allows probabilities to *increase* without limit for positive advantages, concentrating probability mass. Net effect: entropy decreases.
- **Clip-low** (capping ρ_t for positive advantages, $\hat{A}_t > 0$): prevents probabilities from *increasing* too fast. This limits concentration. Net effect: entropy increases.

Under standard symmetric clipping ($\epsilon = 0.2$), the clip-high entropy-reducing effect dominates. Entropy decreases even with random rewards (Engstrom et al., "Clip-Low Increases Entropy, Clip-High Decreases Entropy," 2025).



Symmetric clipping's net effect: clip-high allows probability increases without bound for positive advantages, concentrating mass. Entropy decreases.

Work through: if clip-high is the root cause, what does asymmetric clipping ($\epsilon_{\text{low}} = 0.2$, $\epsilon_{\text{high}} = 0.1$) do to the entropy trajectory?

Token-level analysis (arXiv:2510.10150, "Rethinking Entropy Interventions," 2025): entropy change per update is governed by four factors: clipping strategy, advantage sign, token probability, and token entropy. This gives a precise diagnostic: you can predict which tokens will lose entropy fastest.

Waste pattern. Every step after collapse begins is wasted compute. In long runs, collapse might begin at step 3000 but not be noticed until step 5000. Recovery is difficult: Adam's moment estimates have adapted to the collapsed regime, carrying stale curvature information. Entropy collapse means the model stops exploring, rollouts become homogeneous, and training signal degrades regardless of how much throughput Chapter 3 delivers.

Cost impact. Early detection by 500 steps saves 500× full step cost. At scale, this can exceed the savings from all other levers combined.

Systems reality. Detection must be cheap and non-interrupting:

- Policy entropy over the training batch: free (logits already computed).
- Effective rank of logit matrix (via SVD): expensive for large vocab. Log every 50–100 steps to amortize.
- Reward std across rollouts ($\sigma_R \to 0$ means identical outputs): cheap, already available.
- Top-1 token probability (mean over batch): cheap proxy for determinism.

Intervention options with different cost profiles:

- **Increase entropy bonus:** free. May not work if collapse is deep.
- **Revert to checkpoint N steps before onset:** wastes N steps but guarantees recovery.
- **Asymmetric clipping:** increase ϵ_{low}, decrease ϵ_{high}. Free, addresses root cause per the mechanism literature.
- **Partially reset Adam moments** for output layer parameters: moderate cost, targeted.

Literature:

- "The Entropy Mechanism of RL for Reasoning LMs" (arXiv:2505.22617, PRIME-RL, 2025): sharp early entropy drop leads to performance saturation.
- "Clip-Low Increases Entropy, Clip-High Decreases Entropy" (OpenReview, 2025): clipping mechanism itself causes entropy reduction.
- "Rethinking Entropy Interventions in RLVR" (arXiv:2510.10150, 2025): four-factor token-level entropy change model.
- "Flexible Entropy Control in RLVR" (arXiv:2602.09782, 2026): gradient-preserving entropy control methods.

Exercise: Validating the Clipping-Entropy Mechanism

Part 1: Reproduce the finding. Train GRPO on TinyRL-Code (Chapter 5, 124M) with standard symmetric clipping ($\epsilon = 0.2$). Log at every step: policy entropy (mean over batch), reward std across rollouts, top-1 token probability, gradient norm. Log effective rank every 50 steps.

Part 2: Asymmetric clipping. Train with $\epsilon_{\text{low}} = 0.2$, $\epsilon_{\text{high}} = 0.1$. Compare the entropy trajectory against Part 1. Does asymmetric clipping slow or prevent entropy collapse? What happens to final reward? Does preserving entropy help or hurt?

Part 3: Automated detection and intervention. Implement threshold-based detection: trigger when entropy drops below its 200-step moving average by > 2 standard deviations. Compare two interventions:

(i) Switch to asymmetric clipping ($\epsilon_{\text{low}} = 0.2$, $\epsilon_{\text{high}} = 0.1$)
(ii) Revert to checkpoint 200 steps before the trigger

Which recovers faster? Which reaches higher final reward?

Analysis. Which metric first signals collapse onset? How many steps of warning does the earliest signal give before entropy drops below 0.1?

What to investigate next. The four-factor model from "Rethinking Entropy Interventions" predicts which tokens lose entropy fastest. Can you build a targeted intervention that adjusts clipping only for high-risk tokens rather than globally?

Reward Model vs. Rule-Based Verification

Everything above assumes the reward signal is cheap. That assumption depends on how you compute it.

Rule-based verification (RLVR). A deterministic function checks whether the rollout's output is correct: does the generated code pass unit tests, does the math answer match the ground truth? Cost per rollout: negligible, a string comparison or test execution, orders of magnitude cheaper than a model forward pass. The constraint: you need tasks with verifiable answers. This limits the task distribution to math, code, formal reasoning, and similar domains with ground-truth signals.

Learned reward model. A separate neural network scores each rollout. Cost per rollout: one additional forward pass through the reward model. For a reward model that is a fraction of the policy's size (common: 1/4 to 1/2), this adds 10–20% to the per-rollout cost in Chapter 3's generation pipeline. For a reward model the same size as the policy, it roughly doubles the cost. This directly increases the cost denominator of the north star metric (reward per GPU-second).

The tradeoff. Rule-based verification is cheap but constrains what you can train on. Learned reward models are expensive but unlock open-ended tasks (creative writing, general instruction following, helpfulness). The decision changes the economics of the entire pipeline:

- With rule-based verification, rollout generation dominates cost (60–80%, as shown above). Optimizing Chapter 3's generation throughput is the highest-leverage investment.
- With a learned reward model, the reward computation share grows from 5–10% to 15–30% or more. Now reward model inference competes with generation for optimization attention. You may want to quantize the reward model, batch reward scoring, or use a smaller reward model, each with its own accuracy-cost tradeoff.
- Learned reward models can be wrong. When the reward model has systematic biases, the policy exploits them (reward hacking). This makes KL regularization (Lever 2) more important: the KL penalty is your main defense against the policy drifting into reward-model blind spots.

Ablation as Core Experimental Skill

The four levers above are not things to memorize but things to test. The core skill this chapter teaches is ablation: systematically isolating whether a design choice is load-bearing by removing or varying it and measuring the impact.

How to design an ablation. Every ablation has three components:

1. **Hypothesis.** A specific, falsifiable claim about what a component does. "KL penalty prevents reward hacking" is too vague. "Removing the KL penalty causes held-out reward to diverge from training reward within 500 steps at 124M scale" is testable.
2. **Informative scale.** The smallest experiment that can distinguish the hypothesis from the null. If you are testing whether KL penalty matters, you do not need a 70B model. Run two TinyRL experiments at 124M: one with $\beta = 0.04$, one with $\beta = 0$. If the gap is negligible at small scale, it is likely negligible at large scale (most algorithmic effects are scale-invariant in direction, even if they differ in magnitude). If the gap is large, you have evidence before committing large compute.
3. **Decision-relevant metric.** Measure policy improvement per dollar (reward/GPU-second), not just final reward. A lever that improves final reward by 2% but costs 30% more compute is not load-bearing for cost efficiency.

Example: ablating KL penalty. Run two TinyRL experiments: $\beta = 0.04$ vs. $\beta = 0$, both at 124M for 2000 steps. Measure reward/GPU-second and held-out reward divergence. If $\beta = 0$ matches on both metrics, KL penalty is not load-bearing at this scale and you can drop the reference model forward pass, saving 5–10% of step time and freeing HBM for larger batch sizes in Chapter 3.

Exercise: Ablation Design for Each Lever

Design a cheap ablation for each of the four levers. For each, state:

1. **Hypothesis:** what specific claim you are testing.
2. **Informative scale:** model size, step count, and number of runs needed.
3. **What result would change your Chapter 3 configuration:** if the ablation shows X, how does that change your batch size, generation throughput target, or actor/learner ratio?

Lever 1 (Rollout allocation): Does adaptive G outperform fixed $G = 8$ on reward/GPU-second? If yes, Chapter 3 must support variable-size rollout batches per prompt.
Lever 2 (KL regularization): Does removing KL penalty degrade held-out reward within 2000 steps? If no, drop the reference model to free HBM.
Lever 3 (Credit assignment): Does GRPO-λ reach 80% solve rate at lower G than standard GRPO? If yes, Chapter 3 can generate fewer rollouts per prompt while maintaining the same learning rate.
Lever 4 (Collapse detection): Does asymmetric clipping ($\epsilon_{\text{low}} = 0.2$, $\epsilon_{\text{high}} = 0.1$) prevent entropy collapse that symmetric clipping ($\epsilon = 0.2$) hits? If yes, the training run is more robust and you waste fewer steps on recovery, reducing total generation cost.
Run at least one of these ablations. Report: hypothesis, result, and whether the result changes a decision you made in Chapter 3.

Monitoring the Training Loop

The four levers are not independent knobs. Rollout allocation interacts with credit assignment (poor credit inflates required G). KL scheduling interacts with collapse detection (too-loose β triggers the failure that Lever 4 catches). This section ties them together through what to monitor during a training run.

Minimum metrics to log at every step:

Metric	What it tells you	Lever
Reward / GPU-second	Overall cost efficiency	All
Mean reward (train)	Learning progress	—
Mean reward (held-out)	Generalization / reward hacking	2, 4
Reward std per prompt	Dead rollout rate	1
Policy entropy (mean)	Collapse proximity	4
KL from reference	Regularization regime	2
Gradient norm	Training stability	3, 4
Top-1 token prob	Determinism proxy	4

Failure modes beyond entropy collapse.

- **Reward over-optimization.** Held-out reward diverges from training reward. The policy is specializing to the training prompt distribution or exploiting verifier-specific patterns. *Signal:* training reward rises while held-out reward plateaus or drops. *Intervention:* increase β, diversify prompt set.
- **Length gaming.** Completions grow longer without improving quality, and reward per token drops. The policy discovers that longer outputs score slightly higher due to verifier artifacts. *Signal:* mean completion length increases while reward/token decreases. *Intervention:* add length penalty to reward, or normalize reward by completion length.
- **Gradient explosion / NaN.** Triggered by rare high-reward outliers with extreme log-probabilities.

A single rollout with reward 10× the batch mean creates a gradient spike that destabilizes training. *Signal:* gradient norm spikes by $> 10\times$ its moving average. *Intervention:* gradient clipping (already standard), reward clipping, or outlier filtering in the advantage computation.

- **Straggler amplification.** In distributed rollout generation, one slow completion (e.g., a prompt that elicits a very long response) holds up the entire batch. With $G = 16$ and 100 prompts, one 4096-token completion among 1600 that average 512 tokens wastes GPU-seconds across all workers. *Signal:* high variance in per-prompt generation time. *Intervention:* generation length caps, asynchronous rollout collection, or padding-aware batching. See Chapter 5 for the distributed systems perspective.

Logging cost as compute allocation. Not everything should be logged every step. Effective rank requires SVD, which is expensive for large vocabularies. Log every 50–100 steps. Held-out reward requires inference on a separate prompt set; log every 200–500 steps. Dashboard design is itself a cost decision.

The RL Pipeline

Why This Matters at Frontier Scale

You have the pieces. Chapter 1 taught you where memory bottlenecks live. Chapter 2 taught you how to diagnose and optimize kernels. Chapter 3 taught you how to build a rollout generation pipeline that saturates hardware. Chapter 4 taught you what the RL algorithm demands from those rollouts: group size, advantages, KL constraints, collapse detection. This chapter composes everything into a single system and gives you a platform to test it.

The gap between "I understand each component" and "I can reason about the whole pipeline" is where most engineering effort is wasted. Batch size in rollout generation affects advantage variance in the policy update. Quantization in the actors affects reward signal quality. The actor/learner ratio determines whether your pipeline is generation-bound or update-bound. You cannot make informed local decisions without seeing the full loop.

Move 1: The Pipeline as a Single System

The RL training loop is not a sequence of independent stages. It is a feedback loop where every stage constrains every other. This section walks through the complete data flow, then shows where the constraints bind.

Think First (no computer): Draw the Loop Before Reading

On paper, draw the complete RL training loop from "sample a prompt" to "updated weights." Label every data artifact that flows between stages (prompts, completions, rewards, advantages, gradients, new weights). For each artifact, write the shape: how many prompts, how many rollouts per prompt, how many tokens per rollout. Now: which stage produces the largest tensor? Which stage is the bottleneck? Check your answers against the walkthrough below.

Stage 1: Prompt Sampling

Select B prompts from the training distribution. The distribution matters: curriculum effects are real. Prompts that are too easy produce zero-variance advantages (all rollouts succeed, $\hat{A}_i \approx 0$, no gradient signal). Prompts that are too hard produce zero-variance advantages for the opposite reason (all rollouts fail). The informative prompts are at the policy's current capability frontier, where some rollouts succeed and some fail.

Data artifact: B prompt strings. Typical B: 64–256.

Stage 2: Rollout Generation

For each prompt, generate G completions using the current policy π_θ. This is Chapter 3's optimization target: maximize tokens per second per dollar under the constraints that matter for training (signal quality, not user latency).

Every design decision from Chapter 3 applies here:

- **Batch size.** Larger batches improve GPU utilization (Chapter 2's arithmetic intensity) but increase memory pressure and can delay updates.
- **Quantization.** INT8 or FP8 actors generate tokens faster, but reduced precision introduces noise into the action distribution. The policy update (Stage 5) sees this noise as reward variance.
- **KV-cache management.** Prompt prefixes shared across G rollouts for the same prompt can

share KV-cache entries (Chapter 3's prefix sharing).
- **Sequence length.** Generation length caps trade off between allowing the model to reason and preventing straggler rollouts that hold up the batch.

Data artifact: $B \times G$ completions, each up to L tokens. At $B = 128$, $G = 16$, $L = 512$: 2048 completions, ∼1M tokens. This is the dominant cost, 60–80% of wall time (Chapter 4's cost breakdown).

Stage 3: Reward Computation

Score each completion. In RLVR: run the verifier (unit tests, math checker, formal proof system). In RLHF: run the reward model forward pass. The reward type determines the signal quality and the failure modes (Chapter 4's RLVR vs. RLHF discussion).

- **Deterministic verifiers** (unit tests, exact-match) are cheap and clean but binary: pass or fail. Partial credit requires engineering (fraction of tests passed).
- **Neural reward models** are continuous-valued but gameable. The reward model is itself a bottleneck: a separate forward pass per completion through a model that may be as large as the policy.

Data artifact: $B \times G$ scalar rewards. The reward tensor is tiny compared to the completions, but its quality determines everything downstream.

Stage 4: Advantage Estimation

Convert raw rewards to per-rollout advantages. GRPO computes group-normalized advantages within each prompt:

$$\hat{A}_i = \frac{R_i - \bar{R}_{\text{group}}}{\sigma_{R,\text{group}} + \epsilon}$$

This is Chapter 4's Lever 1. Key interactions:

- **Group size G controls variance.** Small G gives noisy advantages. Large G gives clean advantages but costs $G\times$ more generation. The optimal G depends on the reward distribution, which depends on the prompt difficulty, which changes as the policy improves.
- **Batch normalization scope.** Normalizing across the entire batch vs. per-prompt changes gradient direction. Per-prompt normalization (GRPO default) ensures every prompt contributes signal regardless of absolute reward magnitude.
- **Dead rollouts.** If all G rollouts for a prompt get the same reward, $\sigma_R = 0$ and advantages are undefined. These prompts contribute no gradient. The fraction of dead-rollout prompts is a direct measure of wasted compute.

Data artifact: $B \times G$ scalar advantages, mean ≈ 0, std ≈ 1 per prompt group.

Stage 5: Policy Update

Compute the clipped policy gradient and update weights. This is Chapter 4's clipping, KL penalty, and entropy bonus:

$$\mathcal{L}_{\text{clip}} = -\mathbb{E}\left[\min\left(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right] + \beta \cdot D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}})$$

The backward pass is fast relative to generation (5–15% of wall time). But the update determines

how much the policy changes, which determines how stale the actors' weights become, which determines how many useful rollouts the next batch produces.

Key interaction: staleness. If actors use weight version θ_t but the learner has already updated to θ_{t+k}, the rollouts are off-policy by k steps. The importance sampling ratio $r_t(\theta) = \pi_{\theta_{t+k}}(\tau)/\pi_{\theta_t}(\tau)$ deviates from 1, and clipping activates more aggressively. In the limit, all rollouts are clipped to zero effective gradient. This is why actor/learner pipelining (Chapter 3) must be co-designed with the clipping threshold (Chapter 4).

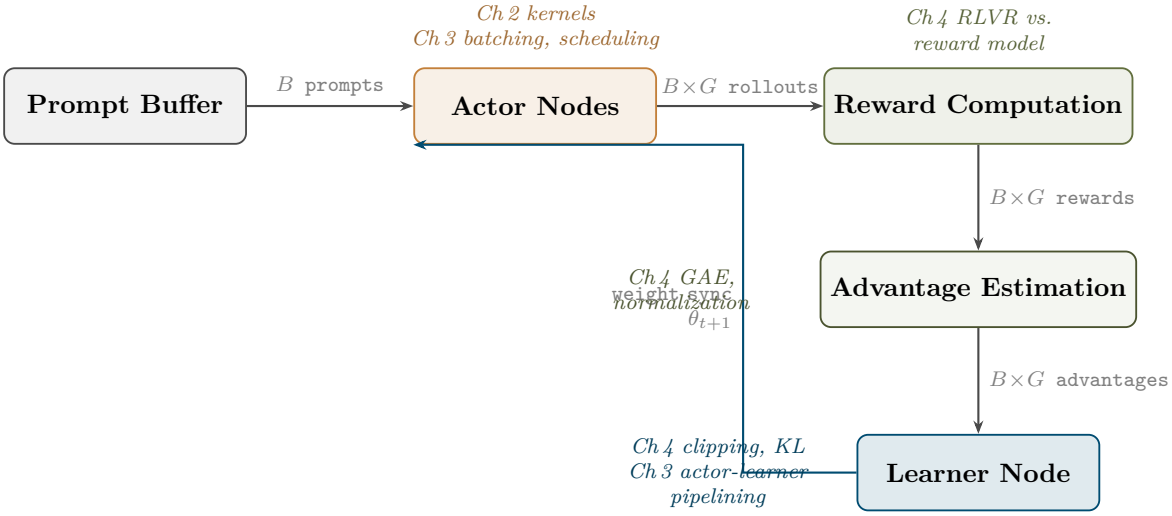
Data artifact: Updated weight tensor θ_{t+1}. Size: the full model. At 1B parameters in FP16: 2 GB that must be broadcast to all actors.

Stage 6: Weight Sync and Repeat

Broadcast θ_{t+1} to all actors. The sync latency determines the minimum staleness of the next batch. With N actor nodes:

- **Synchronous sync:** All actors wait for the new weights. Zero staleness, but the pipeline stalls during broadcast. Wall time = generation + update + sync.
- **Asynchronous sync:** Actors continue generating with old weights while the learner updates. Higher utilization, but staleness accumulates. The clipping threshold ϵ implicitly bounds how much staleness is tolerable.

The new policy generates new rollouts. The loop repeats.



The complete RL training loop as data flow. Each stage is annotated with the chapter whose optimizations apply. The feedback arrow from learner to actors closes the loop: the new policy generates new rollouts, and the cycle repeats.

Where the Constraints Bind

The stages are not independent knobs. Here are the interactions that catch people:

1. **Batch size × advantage variance.** Larger batches in Stage 2 reduce advantage noise in Stage 4 (more samples → lower variance of the group mean). But larger batches mean longer generation time per step, which means the policy changes more between batches, which means the rollouts in Stage 2 are more stale relative to the learner in Stage 5. You cannot optimize batch size without reasoning about both effects simultaneously.
2. **Actor quantization × reward signal quality.** FP8 actors (Stage 2) generate tokens 1.5–2× faster than FP16 actors, but the action distribution is noisier. This noise propagates through Stage 3 (the reward reflects the noisy actions, not the FP16-equivalent actions) and Stage 4 (advantage estimates are computed over noisier rewards). The reward signal-to-noise ratio determines whether the throughput gain is worth the signal degradation.
3. **Actor/learner ratio × staleness.** More actors relative to learners means higher aggregate throughput in Stage 2 but longer intervals between weight syncs in Stage 6. If you have 8 actor nodes generating rollouts while 1 learner node computes updates, the learner processes one batch while actors generate the next. But if actor throughput exceeds learner throughput, rollouts queue up, and by the time the learner processes them the weights have moved. The clipping threshold ϵ (Stage 5) implicitly caps the tolerable staleness.
4. **Reward cost × pipeline balance.** Neural reward models in Stage 3 require a separate forward pass that may rival the generation cost. If reward computation is slow, the pipeline stalls between Stages 2 and 4, and generation hardware idles while rewards are computed. Verifiable rewards (unit tests, math checkers) are nearly free, which is one reason RLVR pipelines achieve higher hardware utilization than RLHF pipelines.

Exercise: Pipeline Bottleneck Analysis

You have the following configuration:

- 4 actor nodes (FP8, 50k tokens/sec each)
- 1 learner node (FP16, processes 200k-token batches)
- $B = 128$ prompts, $G = 16$ rollouts/prompt, avg completion length $L = 256$ tokens
- Reward: unit tests, negligible cost
- Learner update: 8 seconds per batch
- Weight sync: 2 seconds

(a) Calculate: total tokens per batch, generation time, total step time. Is this pipeline generation-bound or update-bound?
(b) You can either add 2 more actor nodes or switch actors to FP16 (25k tokens/sec but cleaner signal). Which do you choose, and what additional information would you need to decide?
(c) Your team proposes doubling G to 32 for better advantages. What happens to generation time? To dead-rollout rate? Is this worth it?

Move 2: TinyRL as the Experimental Platform

Understanding the pipeline diagram is necessary. Testing your understanding is what makes it stick. TinyRL is the platform for that testing, small enough to iterate in minutes, rich enough to exhibit the phenomena that matter at scale.

Design Principles

The TinyStories paper (arXiv:2305.07759) showed that language model phenomena (memorization, generalization, induction heads, grokking) are observable at toy scale *if the environment is designed carefully.* Four properties made it work:

1. **Controlled data distribution.** The input distribution is known and manipulable, not scraped from the internet.
2. **Tasks learnable by small models.** GPT-2 scale (<350M parameters, often <10M) is sufficient to exhibit the phenomena of interest.
3. **Clean evaluation.** No ambiguity about what "good" means. Deterministic verification where possible.
4. **Phenomena observable at small scale.** The dynamics you care about (reward hacking, entropy collapse, KL growth) are visible without scaling to billions of parameters.

TinyRL maps each of these to the RL pipeline:

1. **Curated prompt sets.** 50–100 tasks per environment, sampled reproducibly. The training distribution is known.
2. **GPT-2 scale models.** <10M parameters. Train on a single GPU in under an hour. Cheap enough to ablate every hyperparameter.
3. **Deterministic verification.** Unit tests pass or fail. Sentiment scores are computed by a fixed classifier. Inference cost is measurable. No human raters.
4. **RLVR phenomena in minutes.** Reward hacking, entropy collapse, emergent strategies, KL divergence growth, all observable before lunch.

The reframing from Chapters 1–4: TinyRL is not a standalone framework. It is the *experimental platform for testing the decisions from the previous four chapters.* When you change the batch size, quantization strategy, group size, or KL penalty, you run the experiment on TinyRL and measure whether policy improvement per compute dollar went up or down.

Think First (no computer): What Makes a Good Minimal RL Environment?

Before reading the environment descriptions below, design the simplest RLVR environment you can imagine that still exhibits *verification hacking*. Specify: the model size, the task format, the verifier, and the exploit you predict the model will find. How would you know the model is gaming the verifier rather than solving the task? Write this down before continuing.

TinyRL-Code: "The MNIST of RLVR"

The headline environment. RLVR is the frontier paradigm. Code verification is the core RLVR domain at every major lab. If you build one environment, build this one.

Setup:

- **Model:** GPT-2 scale (<10M parameters), trained from a pretrained checkpoint.
- **Task bank:** 50–100 curated programming puzzles, graded by difficulty:
  - *Easy:* implement a single function, verified by 1–3 unit tests (e.g., `reverse_string`, `sum_list`).
  - *Medium:* implement 2 cooperating functions, verified by 5 unit tests (e.g., a parser and a formatter).
  - *Hard:* implement an algorithm with edge cases, verified by 10 unit tests (e.g., BFS on an adjacency list, handling empty graph and disconnected nodes).
- **Reward:** Fraction of unit tests passed. All pass: $r = 1.0$. None pass: $r = 0.0$. Partial credit for intermediate passes.

This is RLVR in miniature. The verifier is objective, rule-based, and in principle non-gameable. In practice, models find exploits:

- **Hardcoded outputs.** The model memorizes test inputs from repeated exposure and returns

hardcoded outputs that pass the tests without implementing the function. *Detection:* held-out test inputs with different values.
- **Format gaming.** The model wraps a trivially wrong answer in `try/except` blocks to avoid exceptions, earning partial credit without solving the task. *Detection:* reward on held-out tests drops while training reward rises.
- **Specification ambiguity.** The model exploits underspecified edge cases in the problem statement that the test suite does not cover. *Detection:* human review of high-reward completions that look suspicious.

These are the same exploit classes that frontier RLVR teams encounter at scale. TinyRL-Code makes them visible, characterizable, and patchable at toy scale.

The Reward Hacking Gallery. Every exploit discovered during training is documented: the completion, the step count at which it emerged, the test suite it gamed, and the fix. This gallery is the most pedagogically valuable artifact TinyRL produces. Every hardcoded output, every format hack, every specification exploit is a concrete instance of Goodhart's Law under verifiable rewards.

Pipeline testing: TinyRL-Code is the environment where batch size, group size, and clipping threshold decisions from Chapters 3–4 matter most. Binary rewards (pass/fail per test) create high-variance advantages. The optimal G to get clean gradient signal depends on puzzle difficulty. This is where you test whether your pipeline configuration produces capability improvements efficiently.

TinyRL-Align: "The RLHF Baseline"

Setup:

- **Model:** GPT-2 scale (<10M parameters), pretrained on text.
- **Prompts:** IMDB movie review prefixes. The model completes the review.
- **Reward:** A pretrained sentiment classifier scores each completion. Positive sentiment = high reward.

The reward is a neural network, not a deterministic verifier. It can be gamed. The policy does not need to write a genuinely positive review; it needs to write text that the classifier scores highly. These are not the same thing.

RLHF dynamics exhibited:

- **KL divergence growth.** The policy drifts from the reference model rapidly when the KL penalty is weak. Observable within 200 steps.
- **Mode collapse.** The policy converges to a narrow distribution of high-reward outputs (verbose, exclamation-heavy text). Entropy collapses.
- **Reward hacking.** Completions scoring near 1.0 from the classifier are often incoherent or nonsensical to a human reader. The gap between proxy reward and true quality is the core RLHF problem.
- **Entropy collapse.** Policy entropy drops steeply as the model converges to its mode-collapsed distribution.

All dynamics visible in under 30 minutes on a single GPU.

Pipeline testing: TinyRL-Align is where the KL penalty coefficient β (Chapter 4, Lever 2) and collapse detection (Chapter 4, Lever 4) matter most. The neural reward provides continuous signal, so advantage estimation is smoother than TinyRL-Code, but the signal is gameable. The contrast with TinyRL-Code shows concretely why the field moved from neural proxy rewards to verifiable rewards, and at what cost.

TinyRL-Quant: "The PE-RL Flywheel"

Setup:

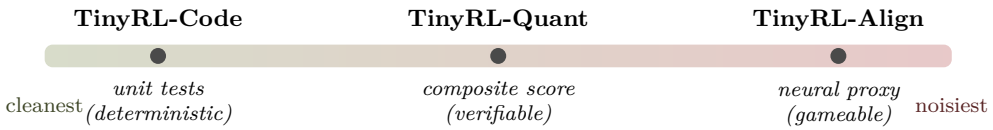
- **Model:** GPT-2 scale (<10M parameters), with per-layer precision as a controllable parameter.
- **Action space:** Choose one of {FP32, FP16, BF16, FP8, INT8, INT4} for each transformer layer. The policy maps layer index and layer statistics to a precision choice.
- **Reward:** r = accuracy $\times$ (1/avg_bits), where accuracy is evaluation quality on a held-out set and avg_bits is the mean bits per weight across layers.

The reward is fully verifiable: accuracy is computed on a fixed evaluation set, and average bits per weight is a deterministic function of the chosen precisions. No neural proxy. But the reward combines two objectives, introducing reward design sensitivity that pure unit-test verification avoids.

Dynamics exhibited:

- **Reward design sensitivity.** The weighting between accuracy and efficiency dramatically changes which layers the policy assigns high precision. Changing the reward formula changes the learned policy.
- **Exploration in discrete spaces.** The precision action space is combinatorial (6^{12} for a 12-layer model). Pure exploitation fails; entropy regularization matters.
- **Eval-set gaming.** If the same evaluation set is used for reward and final assessment, the policy overfits to it. A held-out test set distinct from the reward set is essential.

Pipeline testing: TinyRL-Quant is Chapter 2's quantization tension made concrete. The RL agent *learns to quantize its own layers*. The optimal quantization policy depends on what the model has learned, since RLVR-trained models may concentrate representations differently than SFT-only models, making the optimal per-layer precision assignment differ. This is a testable PE/RL intersection question: run TinyRL-Quant on the same architecture after SFT-only vs. after RLVR training, and compare which layers get high precision.



The verification spectrum. Moving right: reward signal gets noisier, reward hacking shifts from verifier gaming to proxy gaming, and KL regularization becomes more critical.

Think First (no computer): Verification Quality and KL Penalty

Rank the three TinyRL environments by verification cleanliness: TinyRL-Code > TinyRL-Quant > TinyRL-Align. For each, identify the dominant failure mode: verification hacking, KL collapse, or mode collapse. At what point on this spectrum does verification hacking become the dominant failure mode, displacing KL collapse as the primary concern? What does this imply about how verification quality should change your choice of β?

Move 3: The Capstone Exercise

This is the portfolio artifact that composes everything in this book.

Portfolio Project: End-to-End RL Pipeline Optimization

Given a baseline TinyRL-Code configuration and a fixed compute budget:

1. **Profile the baseline.** Where is wall time spent? Measure generation, reward, advantage computation, backward pass, weight sync. Produce a cost breakdown like Chapter 4's stacked bar. Which stage dominates?
2. **Identify the bottleneck.** Use Chapter 1 reasoning (memory-bound vs. compute-bound?) and Chapter 2 reasoning (roofline position, arithmetic intensity) to diagnose *why* the bottleneck stage is slow.
3. **Hypothesize an intervention.** Choose one lever from Chapters 3–4: batch size, quantization, group size G, KL penalty β, clipping threshold ϵ, actor/learner ratio. State which stage it targets and what tradeoff it introduces.
4. **Predict before running.** Write down: "I expect this intervention to change wall time by $X\%$, reward improvement rate by $Y\%$, and the dominant bottleneck will shift from Stage A to Stage B." Quantify. Commit to numbers.
5. **Run at smallest informative scale.** Use TinyRL-Code with the minimum batch size and step count that distinguishes signal from noise. A 50-step run may suffice to measure throughput changes; a 200-step run to measure reward trajectory changes.
6. **Measure: did policy improvement per dollar go up or down?** The metric is not reward alone and not throughput alone. It is reward improvement per unit of compute. A $2\times$ throughput gain that halves reward signal quality is a net loss.
7. **Iterate: what is the next bottleneck?** After your intervention, the bottleneck shifts. Re-profile. The cycle continues.

Deliverable: A documented optimization case study showing the complete reasoning chain: profile → diagnosis → hypothesis → prediction → experiment → comparison with prediction → next iteration. Include at least two full iterations. Format as a publishable blog post or technical report.

Verification criteria:

- Predictions are written down *before* experiments are run, and the document shows where predictions matched and where they diverged.
- At least one intervention improves policy improvement per compute dollar.
- At least one intervention makes things worse, and the analysis explains why.
- The bottleneck shifts at least once across the two iterations.

Research Taste Check

The difference between a good engineer and a great one is the ability to predict which intervention will have the largest effect *before* running the experiment. This exercise tests that skill directly. Anyone can try things and report numbers. The value is in the reasoning chain: why you chose this intervention, what you expected, and what you learned when reality disagreed. That reasoning chain, not the final numbers, is what makes this a portfolio piece.

Exercise: Train Across All Three Environments

For each of TinyRL-Code, TinyRL-Align, and TinyRL-Quant: train with the same algorithm and hyperparameters for 500 steps. For each run, plot reward curve, KL divergence from reference, and policy entropy on the same figure.

Answer: (1) What is the primary pathology in each environment? (2) Which environment shows reward hacking first? (3) In TinyRL-Code, what is the first exploit the model finds? (4) In

TinyRL-Align, at what step does entropy begin to collapse?
Write a one-paragraph characterization of how the verifier type shapes the failure mode. This is the core RLHF-vs-RLVR lesson in one experiment.

Exercise: The KL Removal Experiment

Train each environment for 500 steps with $\beta = 0$ (no KL penalty). State a hypothesis before running.
Key question: Does TinyRL-Code (deterministic verifier) tolerate KL removal better than TinyRL-Align (neural reward)? If RLVR tolerates it better: explain why deterministic verification provides an implicit regularizer that neural rewards do not. If it does not: explain what that tells you about the role of KL in preventing reward hacking versus preventing distribution collapse.
Document: at what KL value does each environment's policy produce visibly degenerate outputs? Check qualitative samples at step 500.

Exercise: Sweep Group Size Under Fixed Compute

Fix total compute (tokens generated per experiment). Sweep $G \in \{4, 8, 16, 32\}$ on TinyRL-Code. As G increases, the number of prompts per batch must decrease to hold total tokens constant.
Measure: (1) final solve rate, (2) advantage variance per prompt, (3) fraction of dead-rollout prompts. Is there an optimal G under fixed compute, or is bigger always better? What does this tell you about the interaction between Stages 2 and 4 in the pipeline?

What this chapter does not cover: production-grade framework engineering (TinyRL is a learning platform, not a deployment system), scaling to thousands of nodes (the principles transfer; the systems engineering does not fit in a chapter), and novel RL algorithms (the pipeline is algorithm-agnostic; swap GRPO for PPO or REINFORCE and the six-stage structure is unchanged). The goal is to make you dangerous with the complete loop at small scale, not to make you an expert in any single stage. The expertise is in the previous four chapters. The integration is here.